

ZSWQ 总体部预研创新类外协课题 开题论证报告

项目名称： 面向异构算网资源的智能感知与预测算法

成果形式： 算法模型类

起止年月： 2024 年 10 月-2025 年 12 月

承担单位： 北京邮电大学

项目负责人： 谢人超

日 期： 2025 年 11 月

目 录

1. 概述	1
2. 研究目标、研究内容与技术指标	1
2.1. 研究目标	1
2.2. 技术指标	3
2.2.1. 功能要求	3
2.2.2. 性能要求	3
2.3. 研究内容	3
3. 研究方案	4
3.1. 设计思路	4
3.2. 设计方案	5
3.2.1. 算法模块组成	5
3.2.2. 算网资源感知子模块	6
3.2.3. 节点任务请求预测子模块	13
3.2.4. 任务需求映射子模块	25
3.2.5. 算法整体输入输出	40
3.2.6. 算法整体流程	43
3.3. 通用质量特性要求	44
3.4. 评估标准	45
3.5. 验证方案	45
3.5.1. 验证要求	45
3.5.2. 训练样本仿真生成	48
3.5.3. 验证项目	48
4. 技术指标满足情况分析	49
5. 研究进度、预期研究成果及应用方向	49
5.1. 研究进度	49
5.2. 预期研究成果	50
5.3. 应用方向分析	50
5.3.1. 典型任务 1：信号级仿真任务	50
5.3.2. 典型任务 2：时敏目标识别任务	50
6. 保障条件及措施	51
6.1. 研发团队保障条件	51
6.2. 保密措施	52
6.3. 质量保障措施	52
7. 承担研究任务主要人员概况	55
8. 技术风险分析	56
9. 经费概算	57

1. 概述

本论证报告供面向异构算网资源的智能感知与预测算法的开发、研制、测试与验收使用，作为算法设计与开发的依据，其中明确了算法的研究内容、技术指标、设计方案和验证方案等内容。

本技术要求涉及的一切内容的使用范围限于双方，没有得到甲方同意，乙方的研究成果不得以任何方式向外界公开或转让。

本算法通过提取节点历史运行信息特征，预测未来节点的任务到达情况，同时构建“任务-算力资源”映射模型用于拟合任务在不同负载体量和类型特征下所需的算网资源需求，并结合对下一时隙任务负载到达情况的预测，赋能系统对未来节点上总体的算网资源需求量的感知能力，从而提高任务执行效率和系统可靠性，解决传统静态资源分配方法在动态环境下的局限性问题。

2. 研究目标、研究内容与技术指标

2.1. 研究目标

当前战场对抗的广度和深度空前扩大，智能化趋势愈发显著，信息化成为现代战场的核心特征。现代战争引入了多样化的算力资源，包括高性能计算节点、分布式计算平台和云边协同架构，以支撑信息化作战任务的高效运行，为现代战争提供强大的技术保障。然而，面向复杂作战场景的分布式算力体系要想实现融合协同与高效运作，主要存在着三方面的以下挑战：

（1）战场异构性

当前作战场景下各级算力泛在部署并具有各异的计算功能，作战单元的物理资源异构性是影响任务执行效率和协同作战能力的关键因素。这种异构性主要体现在节点的物理资源、设备性能、任务负载等方面。首先，各节点配备的 CPU、GPU、内存空间、磁盘空间等计算与存储资源存在显著差异，直接决定了节点的计算能力和任务执行效率的高低。其次，作战任务的到达模式与执行模式具有多样性，业务在类型、优先级上有各异的需求，接入节点不一定适配。

（2）战场高动态化

在现代军事作战中，作战环境具有高动态、强对抗、巨复杂的特点。作战条件的迅速变化和不可预测性进一步加剧了任务负载的动态性。在异构算网资源场景下，瞬间激增的计算需求可能导致某些节点的负载剧烈增加，资源分配压力

骤增，进而影响整体资源利用效率。同时作战任务之间可能存在着紧密的逻辑依赖关系，多数任务的到达并非完全独立，而是由其前置任务的完成触发生成。这增加了对于任务负载量预测的难度。

（3）单节点资源受限

当前战场环境下，资源受限已成为影响作战系统稳定性和任务执行效率的突出问题。现代战场呈现出高对抗性、高动态性和高复杂度的特点，节点所能获取的计算、存储、能耗等资源极为有限。随着任务量的激增与突发事件的频繁发生，节点面临的资源竞争日益激烈，难以及时满足各类作战任务对高性能和高可靠性的迫切需求。因此，如何准确动态感知节点的资源负载趋势，以及时优化资源分配与任务调度，已成为战场作战系统必须解决的核心难题。

为此，本研究将面向异构算网资源场景，构建统一的算力资源感知与预测体系，设计智能化的感知预测算法，为指挥控制系统提供精准的负载预测能力。通过对未来任务负载的精准预测与细粒度的任务资源需求解析，确保任务执行需求与物理资源的适配性，保障复杂作战任务执行的高效性与稳定性。具体研究目标如下：

（1）面向任务的算网需求度量：对任务的细粒度需求进行全面解析，这不仅能够帮助系统更精确地解析和预测不同类型任务对计算资源和网络资源的具体需求，还能显著提升资源分配的合理性和利用效率。本研究旨在以算网任务请求为核心，深入解析任务请求的特征，包括任务执行的时延敏感性、数据需求强度、计算复杂度以及任务引流相关性等多维度信息。通过引入神经网络拟合方式，精准挖掘和解析任务需求，并以统一标准的度量值进行描述，构建任务需求与算力资源之间的高效映射机制，以确保任务能够按照预期的性能和效率高效执行。

（2）面向节点的多维预测：以战场移动性节点的任务负载趋势为核心，深入解析节点运行效能的关键影响因素，包括历史任务的类型信息及作战阶段等多维度数据，通过智能预测算法，挖掘不同特征之间的相关性，以实现对未来时隙中该节点上到达任务请求的精准预测，为系统的资源调度和任务分配提供科学依据。

智能算法可以通过多维特征的相关性提取与建模，兼具预测的准确性和时效性。这种预测能力能够帮助系统提前感知未来任务负载的变化趋势，有效避免

资源分配中的盲目性和滞后性,还可以使系统能够更好地应对复杂环境下的任务高峰和资源瓶颈问题,确保系统在复杂战场环境下能够保障任务的连续性和执行效率。

综上所述,本研究旨在构建一个统一的任务算网资源需求感知与任务负载预测体系的解决方案,以应对信息化作战场景。通过精细化解析任务需求和智能化预测,本方案将助力实现算力资源的全局优化和最佳分配,确保复杂作战任务的高效处理和及时响应,从而为满足未来作战需求提供全面的技术支持。

2.2. 技术指标

2.2.1. 功能要求

- a) 具备对节点未来各项算网资源需求量进行预测的能力。

2.2.2. 性能要求

- a) 算力网络资源负载预测准确率不低于 80%;
- b) 支持不少于 3 种算力网络资源的表征,支撑的算网资源调度管理节点数量不低于 1000 个。

2.3. 研究内容

针对现役指挥系统全局态势感知不足、节点未来状态预测困难和任务资源需求分析缺失的痛点问题,本研究拟设计基于 **Informer** 算法的多维节点负载预测框架,深度分析历史任务运行数据,挖掘任务负载到达的时间相关性与逻辑依赖性,以预测未来时隙任务到达量、类型特征,作为评估节点资源负载变化趋势的基础。同时构建基于多层神经网络与 **LightGBM** 算法的“任务-资源”映射模型,通过细粒度解析任务类型、执行特征等多维属性,实现任务需求与算力资源的精准匹配。通过多维度智能模型的协同优化,系统可动态适配复杂战场环境下的任务与资源关系,增强资源调度的鲁棒性与任务执行效率,为后续研究提供技术支撑。

(1) 研究内容 1: 基于深度 **LightGBM** 算法的任务算网资源需求感知

算法支持对任务请求的多维特征,包括任务类型与任务执行性能需求,进行细粒度解析,利用深度神经网络模型与 **LightGBM** 算法相结合的方式,构建任务需求与算力资源之间的高效映射模型,用于拟合任务在不同规模体量和类型特征下所需的算网资源需求,作为感知任务资源需求的方法。以确保任务服务性能

需求与算网资源需求的高效匹配。同时将“任务-算力资源”映射与任务到达量预测相耦合，实现对下一时隙节点上任务的总体资源需求量进行动态感知，从而提升资源分配效率和任务执行性能。

（2）研究内容 2：基于 Informer 模型的任务请求到达量预测

针对高动态作战场景下节点负载不均的问题，采用 Informer 算法深入提取任务历史任务到达信息、节点历史作战环境等多维数据的关键特征，挖掘任务负载到达量的时间相关性与逻辑依赖性，精准预测未来时隙中的任务到达模式变化趋势，通过高效的预测能力，赋能系统对未来节点上总体的算网资源需求量的感知能力，从而提高任务执行效率和系统可靠性。

3. 研究方案

3.1. 设计思路

面向异构算网资源的智能感知与预测算法旨在解决异构算网资源管理中任务需求模糊、资源分配效率低、动态环境适应性差等问题，核心思路包括以下几点：

（1）战场要素全面感知：通过对节点战场态势以及任务执行需求的全面感知，构建统一的建模和标准化描述，为节点未来任务请求到达量的预测提供高质量的数据支持。

（2）节点任务请求到达量预测：针对高动态作战场景下节点负载不均的问题，采用 Informer 算法深入提取历史任务到达序列信息中任务负载到达量的时间相关性与逻辑依赖性，以精准预测未来时隙节点上将到达任务的总体负载量与综合类型特征。通过高效的预测能力，赋能系统对未来节点上总体的算网资源需求量的感知能力，从而提高任务执行效率和系统可靠性。

（3）任务总体算网资源需求分析：通过将任务资源需求解析与任务负载量预测相结合，实现对节点资源负载变化趋势的合理评估。采用动态更新机制，基于实时任务执行情况对“任务-资源”映射模型进行定期优化，持续学习任务资源需求的变化，确保系统能够适应动态环境下的任务需求，进一步提升资源分配效率和任务执行性能。

算法设计思路如图 1 所示，其中节点任务请求量预测以战场要素的全面感知为基础，实现对未来负载到达情况的精准预测，并将预测结果作为映射模型提

供关键输入，从而动态感知节点上算网资源需求变化趋势。“任务-资源”映射模型负责将任务的多维特征转化为具体的算网资源需求。面向异构算网资源的智能感知与预测算法通过“感知-预测-映射”的耦合机制，可提升任务与资源的匹配效率和系统的动态适应性，实现算网资源的全局统筹与最优分配，确保任务执行的高效性和稳定性。

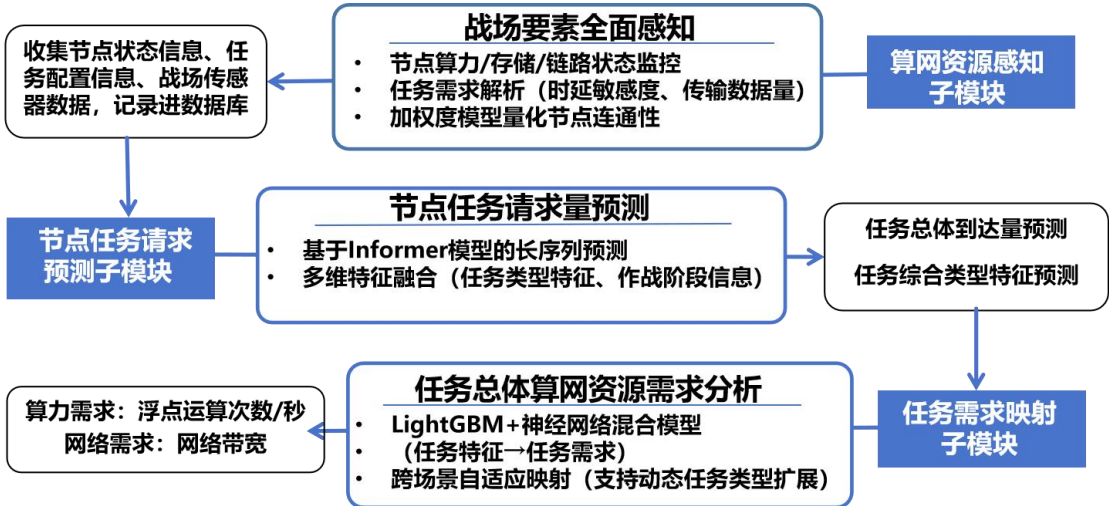


图 1. 算法设计逻辑图

3. 2. 设计方案

3. 2. 1. 算法模块组成

面向异构算网资源的智能感知与预测算法的总体架构图如图 2 所示。该算法接收由算网资源管理软件任务配置模块输入的任务执行特征信息以及由车载传感器输入的战场态势信息，根据两种不同的算法调用模式输出节点下一时隙的总体算网资源需求或实时任务的资源需求信息。

面向异构算网资源的智能感知与预测算法由三个主要子模块构成：算网资源感知子模块、节点任务请求预测子模块和任务需求映射子模块。算网资源感知子模块负责对战场作战阶段以及任务执行需求进行感知，并将相关的数据存入数据库中作为后续算法的输入。节点任务请求预测子模块接收数据库中历史日志时序数据，这些时序序列数据包括了各类型任务历史负载到达序列、战场特征信息。利用 Informer 模型，该子模块能够挖掘出任务负载到达量与任务依赖逻辑的时间相关性，并对下一时隙将到达的任务综合负载情况和节点的负载变化趋势进行准确预测评估。在时间触发模式下，任务需求映射子模块接受节点任务请求预测子模块输出的任务到达情况预测值，利用“任务-资源”映射模型进一步解析出节点

上计算资源和网络资源的总体需求。在事件触发模式下，任务需求映射子模块接受实时的任务配置信息，映射执行该任务的算网资源需求。

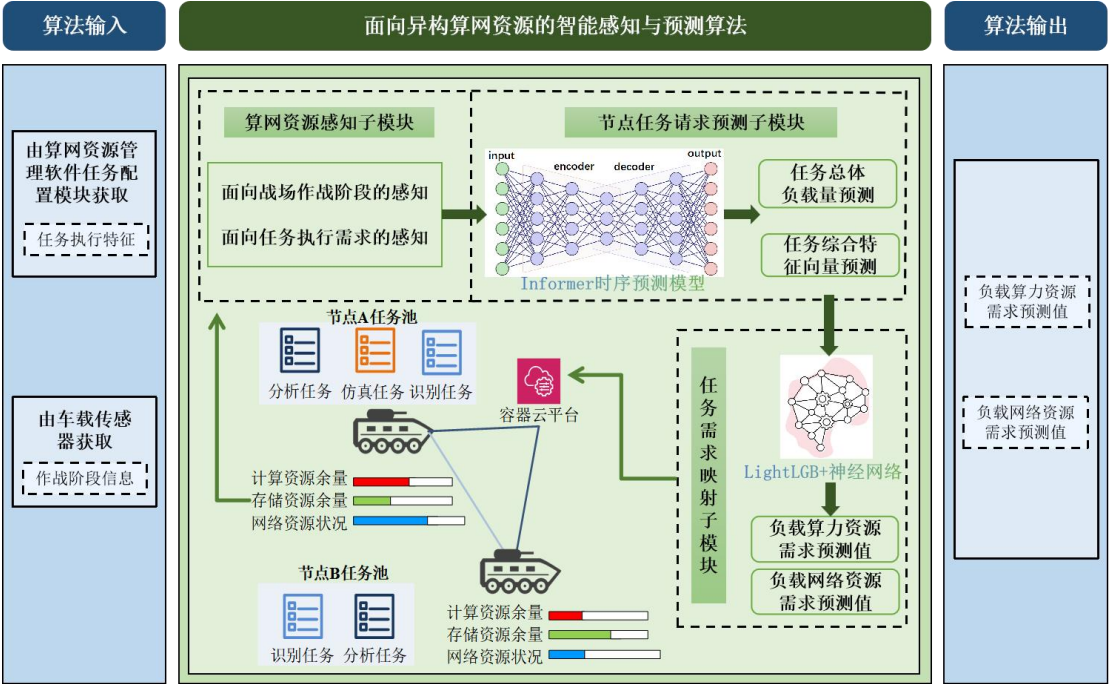


图 2. 算法架构图

3. 2. 2. 算网资源感知子模块

3. 2. 2. 1. 模块功能

算网资源感知模块的主要功能是通过节点战场态势以及任务执行需求的全面感知，构建统一的建模和标准化描述，为节点未来任务请求到达量的预测提供高质量的数据支持。

针对作战任务的多样化需求，模块还需根据任务的执行需求，对任务进行细粒度的类型特征建模，以挖掘不同类型任务的时序到达规律。

3. 2. 2. 2. 面向战场作战阶段的感知

面向战场作战阶段的感知是模块的关键能力，其核心在于准确把握不同作战阶段下任务请求的时空分布特征。随着战场态势的演进，节点接收的任务请求到达量密度与类型呈现出明显的阶段性变化规律。因此，系统需对节点所处的作战阶段进行精准感知，通过车载传感器获取作战阶段划分信息及相应的态势评估结果，为后续的资源动态调整与任务调度提供战场环境上下文，从而实现节点资源与任务需求在时间维度上的最优匹配。

3. 2. 2. 3. 面向任务执行需求的感知

面向任务执行需求的感知是模块的重要功能之一，主要负责接收和解析算网需求度量软件的算网任务配置模块发送的任务配置信息。这些信息包括：

- 任务执行时所需的传输数据量 d_{size} ：描述任务在执行过程中需要传输的数据规模；
- 任务执行时所需的处理数据量 c_{size} ：描述任务在节点上需要处理的数据规模；
- 最大时延容忍值 t_{delay} ：描述任务对执行时延的性能要求。
- 基于概率的任务依赖关系图：描述在任务逻辑链中，该任务与其他任务之间触发概率。

对于任务类型特征进行标准化度量与表征是任务负载预测以及任务资源需求映射的重要依据。为了后续模型能够更好地捕捉与学习任务的类型特征，需要对任务的类型特征进行合理化建模，以揭示不同类型任务潜在的到达模式与到达规律。

任务到达模式与多种任务特征因素均存在潜在关联，为了优化系统性能和制定有效的调度策略，需要对任务到达频率进行深入分析和研究。传统对于任务类型的描述往往采取 one-hot 的编码形式，这种粗粒度的任务类型划分形式过于单一，考虑到任务执行需求的多样化，本方案中将从以下维度对任务类型进行全面的多维向量表征。

（1）时间敏感性（ T ）

时间敏感性描述的是任务对完成时间的严格要求程度。具有高时间敏感性的任务通常需要在特定的时间窗口内完成或者需要实时响应，且往往具有较高的触发频率，以满足实时性需求。例如，在战场环境中，目标识别任务可能每秒需要被触发多次。而时间敏感性较低的任务触发频率较低，且其触发时间分布更为随机。例如，离线数据分析任务可能每天仅触发一次。

（2）数据需求强度（ D ）

数据需求强度的是描述任务在执行过程中需要传输的数据量大小。数据需求强度高的任务通常需要传输大规模数据流，如视频流、传感器数据，其通常具有较高的到达频率，因为数据流的产生是连续的。例如，视频流处理任务可能每秒触发多次以处理实时数据。而数据需求强度低的任务到达频率较低，且较为离

散的。例如，轻量级的战场指令传输任务可能每分钟触发一次。

（3）计算复杂度（ C ）

计算复杂度描述的是任务在执行过程中对计算资源的需求程度。计算复杂度高的任务通常需要大量的算力资源加以支持，其通常到达频率较低。例如，信号级仿真计算任务可能每小时触发一次，且单个执行时间可能长达数个小时。而计算复杂度低的任务到达频率较高，尤其是轻量级任务。例如，简单的数据采集任务可能每秒触发多次。

（4）任务引流关联性（ R ）

任务引流关联性描述的是任务到达之间的条件依赖关系，即一个任务的到达是否可能影响其他任务的到达时间分布。具有高引流关联性的任务通常是触发链条中至关重要的环节，其到达往往引导多个后续任务的出现。例如，在战场环境中，毁伤方案拟制任务之后会触发火力方案筹划任务，之后有一定概率触发方案推演评估任务。而引流关联性较低的任务往往独立发生，与其他任务的到达几乎无关。例如，目标情报处理任务、作战状态监控任务等任务的到达不依赖其他任务，也不会进一步触发其他任务。

基于上述分析，时间敏感性、数据需求强度、计算复杂度和任务引流关联性是影响任务到达频率的核心因素。故选取上述四种任务属性特征，将任务类型建模为一个多维特征向量 Q ，表达式如下：

$$Q = (T, D, C, R)$$

其中 T 代表时间敏感性， D 代表数据需求强度， C 代表计算复杂度， R 代表任务引流关联性。

在任务特征的多维向量表示中，任务类型特征向量的各个特征具有不同的物理意义和数值范围。为了确保模型能够准确处理这些特征，需要对部分特征进行归一化，即需要将任务特征的每个维度的数值范围映射到 $[0,1]$ ，以确保不同特征具有相同的量级，避免因数值范围差异导致模型学习过程中的偏差。在归一化过程中，需根据任务特征的实际意义，对任务的不同特征分别定义归一化处理方式以精准衡量任务不同维度的执行性能特点，同时保持任务特征的相对差异性，使不同任务的特性能够清晰地反映在特征值中，从而使模型能够更快地收敛，同时提高预测精度和泛化能力。

(1) 时间敏感性(T)

任务的时间敏感性由其最大容忍响应时间决定。时间敏感性越高,任务对响应时间的要求越严格,最大容忍响应时间越短。根据任务的最大容忍响应时间 t_{delay} ,将时间敏感性归一化为[0,1]的范围。归一化公式如下:

$$T = 1 - \frac{t_{delay} - t_{min}}{t_{max} - t_{min}}$$

其中:

- t_{delay} 表示任务的最大容忍响应时间。
- t_{min} 表示所有任务中最短的最大容忍响应时间。
- t_{max} 表示所有任务中最长的最大容忍响应时间。

示例:

- 时敏目标识别任务(t_{delay} 为 100ms): $T = 1 - \frac{100-100}{7200000-100} = 1.000$;
- 信号级仿真任务 (t_{delay} 为 2 小时): $T = 1 - \frac{7200000-100}{72000000-100} = 0.000$;
- 战场后勤补给分析任务(t_{delay} 为 1 小时): $T = 1 - \frac{3600000-100}{7200000-100} \approx 0.500$ 。

(2) 数据需求强度(D)

任务的数据需求强度由执行单个任务所需传输的数据量决定。数据需求强度越高,任务需要传输的数据量越大。根据执行单次任务的所需传输的数据量 d_{size} ,将数据需求强度归一化为[0,1]的范围。归一化公式如下:

$$D = \frac{d_{size} - d_{min}}{d_{max} - d_{min}}$$

其中:

- d_{size} 表示任务的数据传输量。
- d_{min} 表示所有任务中最小的数据传输量。
- d_{max} 表示所有任务中最大的数据传输量。

示例:

- 时敏目标识别任务 (d_{size} 为 50MB): $D = \frac{50-10}{2048-10} \approx 0.980$;
- 信号级仿真任务(d_{size} 为 2GB): $D = \frac{2048-10}{2048-10} = 1.000$;
- 战场后勤补给分析任务(d_{size} 为 10MB): $D = \frac{10-10}{2048-10} = 0.000$ 。

(3) 计算复杂度(C)

任务的计算复杂度由执行单个任务所需处理的数据量决定。计算复杂度越高，任务对计算资源的需求越大。根据任务的单次处理数据量 c_{size} ，将数据需求强度归一化为[0,1]的范围。归一化公式如下：

$$C = \frac{c_{size} - c_{min}}{c_{max} - c_{min}}$$

其中：

- c_{size} 表示任务的单次处理数据量。
- c_{min} 表示所有任务中最小的单次处理数据量。
- c_{max} 表示所有任务中最大的单次处理数据量。

示例：

- 时敏目标识别任务 (c_{size} 为 20MB) : $C = \frac{20-5}{1024-5} \approx 0.985$;
- 信号级仿真任务(c_{size} 为 1GB): $C = \frac{1024-5}{1024-5} = 1.000$;
- 战场后勤补给分析任务(c_{size} 为 5MB): $C = \frac{5-5}{1024-5} = 0.000$ 。

(4) 引流关联性(R)

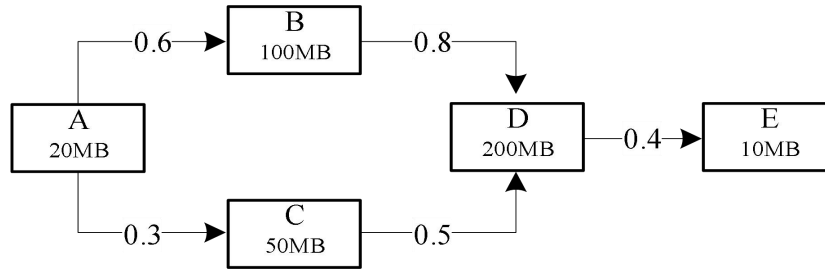


图 3. 概率加权的任务依赖关系图

任务的引流关联性计算基于任务之间的转移概率关系。实际作战环境中的任务链往往只是简单的串行逻辑，为了增强算法的可拓展性，这里假设一条已知的军事行动打击链由 5 个具有关联依赖性的任务组成，其依赖关系构成有向无环图（DAG），如图 3 所示。任务之间的触发用有向边表示，边的权重代表触发概率。任务 A 触发任务 B 的概率为 60%，即 $P(A \rightarrow B) = 0.6$ 。任务 A 的单次处理数据量为 20MB。任务的引流关联性值 $R(T_i)$ 反映了任务 T_i 通过直接和间接路径触发下游任务的综合能力。计算时设置最大深度 k ，即考虑从任务 T_i 出发不超过 k 跳的路径。同时，引入衰减系数以体现路径递增跳数导致的贡献减弱：

任务的引流关联性计算公式如下：

- (1) 任务的直接引流关联性是当前任务通过第一跳，对直接连接的下游任务贡献的总概率值。直接引流关联性公式为：

$$r_{direct}(T_i) = \sum_{T_j \in Out(T_i)} P(T_i \rightarrow T_j) \times c_{size}(T_j) \times \lambda_1$$

其中：

- $Out(T_i)$ 表示当前任务 T_i 的直接下游任务集合。
- $c_{size}(T_j)$ 表示后续任务 T_j 的单个处理数据量。
- $P(T_i \rightarrow T_j)$ 表示任务 T_i 到任务 T_j 的转移概率。
- λ_1 表示直接路径的衰减系数，直接路径的贡献通常不衰减，因此默认值为 1.0。

- (2) 任务的间接引流关联性是当前任务通过多跳路径，对下游任务触发的累计贡献。间接引流关联性公式为：

$$r_{indirect}(T_i) = \sum_{Path \in \mathcal{P}_{2,k}(T_i)} \left(\prod_{[T_j \rightarrow T_s] \in Path} P(T_j \rightarrow T_s) \times c_{size}(T_s) \times \lambda_2^{\Delta-1} \right)$$

其中：

- $\mathcal{P}_{2,k}(T_i)$ 表示从任务 T_i 出发，所有路径长度为 2 跳至 k 跳的路径集合。
- $c_{size}(T_s)$ 表示后续任务 T_s 的单个处理数据量。
- $e \in Path$ 表示路径中所有任务间的链接关系。
- $P(e)$ 表示边 e 所代表任务之间的触发概率。
- λ_2 表示间接贡献的单步衰减系数，默认为 0.8（间接路径触发，贡献随跳数衰减）
- Δ 表示路径的跳数（即路径包含的边数）。

- (3) 任务的总引流关联性值由直接引流关联性和间接引流关联性两部分组。总引流关联性公式为：

$$r_{total}(T_i) = r_{direct}(T_i) + r_{indirect}(T_i)$$

其中 $r_{direct}(T_i)$ 反映了任务 T_i 第一跳的贡献， $r_{indirect}(T_i)$ 反映了任务 T_i 通过更长路径对任务链产生的深远影响。

示例：以任务 A 的引流关联性计算为例，设最大关联深度 $k=3$ 。

(1) 直接引流关联性（1 跳）：

$$r_{direct}(A) = 0.6 \times 1.0 \times 100 + 0.3 \times 1.0 \times 50 = 75$$

(2) 间接引流关联性（2-3 跳）：

- 路径 (A→B→D)： $0.6 \times 0.8 \times 200 \times 0.8 = 76.8$
- 路径 (A→C→D)： $0.3 \times 0.5 \times 200 \times 0.8 = 24$
- 路径 (A→B→D→E)： $0.6 \times 0.8 \times 0.4 \times 10 \times 0.8^2 = 1.2288$
- 路径 (A→C→D→E)： $0.3 \times 0.5 \times 0.4 \times 10 \times 0.8^2 = 0.384$

因此任务 A 的总间接引流关联性为：

$$r_{indirect}(A) = 76.8 + 24 + 1.2288 + 0.384 = 102.4128$$

(3) 任务 A 的总引流关联性：

$$r_{total}(T_i) = r_{direct}(T_i) + r_{indirect}(T_i) = 75 + 102.4128 = 177.4128$$

同理可得其他任务的引流关联性结果：

任务	$r_{direct}(T_i)$	$r_{indirect}(T_i)$	$r_{total}(T_i)$
A	75	102.4128	177.4128
B	160	2.56	162.56
C	100	1.6	101.6
D	4	0	4
E	0	0	0

通过明确拆分任务对后续任务直接和间接引流影响，解决了混合作用的模糊性，并通过衰减系数和深度限制来调整计算范围和权重，提供了更细粒度的任务依赖关系描述。计算得到任务的引流关联性 $r_{total}(T_i)$ 后，将引流关联性归一化为[0,1]的范围。归一化公式如下：

$$R = \frac{r_{task} - r_{min}}{r_{max} - r_{min}}$$

其中：

- r_{task} 表示当前任务的引流关联性。
- r_{min} 表示所有任务中最小的引流关联性。
- r_{max} 表示所有任务中最大的引流关联性。

综上，将任务类型特征向量描述的具体指标与每个维度指标的归一化处理方法总结成任务类型特征向量表，具体内容见表 1。

表 1 任务类型特征向量表

任务类型特征	赋值方式
时间敏感性(T)	$T = 1 - \frac{t_{max_response} - t_{min}}{t_{max} - t_{min}}$
数据需求强度(D)	$D = \frac{d_{size} - d_{min}}{d_{max} - d_{min}}$
计算复杂度(C)	$C = \frac{c_{size} - c_{min}}{c_{max} - c_{min}}$
引流关联性(R)	$R = \frac{r_{task} - r_{min}}{r_{max} - r_{min}}$

3.2.3. 节点任务请求预测子模块

3.2.3.1. 模块功能

本算法基于 **Informer** 模型，通过分析本地节点的历史运行信息，包括到达任务的请求量、类型特征以及所属的作战阶段，实现对下一时隙节点上综合任务请求到达情况的精准预测。**Informer** 模型优异的长序列建模能力和多特征输入数据的处理能力使其能够有效捕捉任务负载到达与任务依赖触发的时序变化趋势，为系统资源调度提供预见性支持。

3.2.3.2. 模块设计思路

不同作战任务的到达模式与作战阶段、任务类型等可量化特征紧密相关。任务的到达与转移并不是单一模式驱动的，既可能包含时间周期规律，也可能受依赖前驱任务触发影响。考虑到以上多重因素影响，本模块采用 **Informer** 时序预测模型，输入具有多维度特征的历史时序序列，挖掘特征之间的相关性，对未来短时间窗口内的任务总体负载量与任务综合类型特征向量进行高精度预测。

Informer 模型是一种面向长时序数据预测任务的深度学习方法，天然支持多变量输入和多任务输出。它在设计时就是为了解决传统算法在处理大规模、多变量、长跨度时间序列时效率低、内存占用大和预测能力不足的问题。当前作战场景下对于下一时隙任务到达情况的预测，需要结合作战阶段、任务请求量，任务类型特征等多维特征进行深度分析，这正是 **Informer** 模型能够发挥优势的场景。

Informer 的核心结构沿用自 **Transformer**，主要由编码器和解码器两部分组成。编码器可以理解为筛选和压缩历史任务信息，学习并提炼关键的时序特征；解码器会根据编码出的信息，生成未来时间节点任务负载到达情况的预测值。**Informer**

采用稀疏注意力机制，智能选择出当前输入的上下文中对未来影响最大前 K 个的关键历史片段重点学习，并赋予每个时隙数据不同的权重。稀疏注意力机制相比于传统注意力层使得本应指数级增长的注意力矩阵大幅减小，极大的降低了计算复杂度，节省了内存开销。同时多头注意力的设计可以在多个投影空间中表达不同的注意力分布，使模型对不同特征通道赋予不同的关注度，以提升模型对于特征间关系的建模能力。

基于 Informer 模型实现的时序相关的任务负载到达情况流程如图 4 所示：

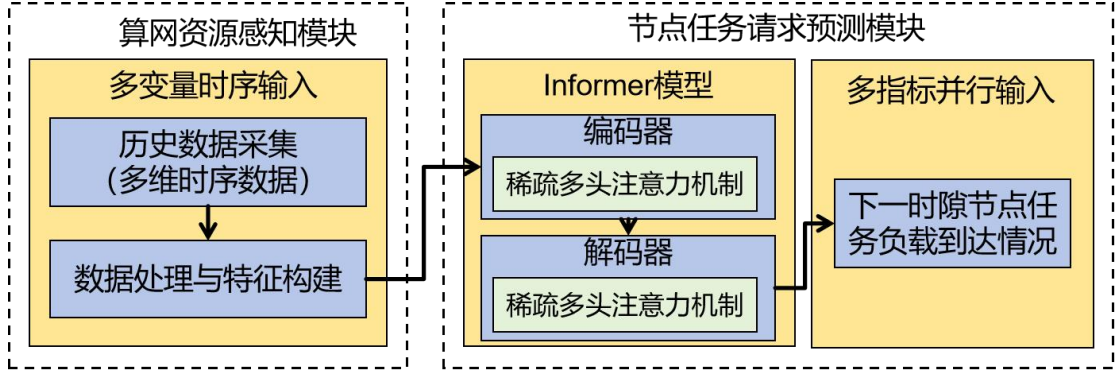


图 4. 基于时序的任务负载预测示意图

3.2.3.3. 算法流程设计

3.2.3.3.1. 任务综合特征表征

由于任务的依赖逻辑关系可能作战环境发生改变，因此需要对每个预测时隙内输入的任务依赖逻辑关系进行判定。当任务依赖逻辑发生变化时，则需要对相应任务的引流关联性属性值进行重新计算，如果未发生改变，则无需计算。

考虑到战场的复杂度，同一时刻可能会到达多种类型的任务。并且在不同的作战区域和作战阶段，到达任务的类型也会发生相应变化。为了提高预测算法的拓展性和自适应性，需要将同时到达的不同类型任务进行抽象化处理，形成一个综合的任务表征，以灵活应对各类任务特征的差异。同时这样处理可以有效降低算法的输入数据维度，减少计算开销，提升算法的执行效率。

从任务综合负载量角度，将同一时刻到达的不同任务对应的负载量求和，得到节点在该时刻需要处理的总数据量，作为任务负载的整体强度指标。假设某一时刻到达节点的任务类型数量为 N ，则任务综合负载量的计算公式为：

$$L = \sum_{n=1}^N l_n$$

其中 L 表示任务综合负载量， l_n 表示第 n 个任务的负载量，即执行该任务所需处理的数据量。

从任务综合类型特征表征角度，通过加权方式来综合从算网资源感知子模块中接受到的不同任务的类型特征，生成一个能够反映当前时刻任务类型分布的表征。由于不同类型的任务之间的负载量可能存在较大的动态范围，比如信号级仿真任务所需执行的数据量远大于目标识别任务，直接使用负载量作为权重可能导致某些任务的特征主导综合表征，故采用对数归一化来压缩负载量的动态范围，之后再根据各个任务规范后的负载量计算任务权重，计算公式如下：

$$\alpha_n = \frac{\log(1 + l_n)}{\sum_n \log(1 + l_n)}$$

其中 α_n 是归一化后的第 n 个任务的权重，表示每个任务在综合表征中的相对重要性。

对任务类型特征向量进行加权表征，生成综合任务类型特征向量，计算公式如下：

$$\mathbf{Q} = \frac{\sum_{n=1}^N \alpha_n \cdot \mathbf{Q}_n}{N}$$

其中 $\mathbf{Q}$ 表示综合任务类型表征向量， N 表示某一时刻到达节点的任务类型数量，

$\mathbf{Q}_n$ 表示第 n 个任务的任务类型特征向量(T, D, C, R)， α_n 是第 n 个任务的权重。

将任务综合负载量和任务综合类型特征向量作为预测算法中关于任务特征的输入部分，以此来代替对于不同类型任务负载量与类型特征向量的独立输入，满足算法的拓展性要求。

3.2.3.3.2. 输入数据特征选取

为了提高预测的准确性和模型的泛化能力，需要综合考虑各类型任务历史负载到达序列、任务类型特征以及任务生成的作战阶段共同作为输入特征。这种多维度特征的引入能够增强模型对复杂模式的捕捉能力，更精准地反映任务请求到达的潜在时间规律。

具体来说，历史任务负载到达量直观反映了时间序列中的趋势性、周期性和突发性等特征信息，有助于捕捉任务负载的时变性。例如，任务负载可能呈现日常周期性波动（如白天高峰与夜间低谷）或突发性的峰值负载（如临时任务的集中到达）。任务类型特征与任务的到达频率密切相关，可以揭示潜在的任务到达

规模。作战阶段则反映了当前的战场态势，随着作战阶段的变化，任务的到达模式也会相应变化。

3.2.3.3. 模型输入格式

为了精准预测在未来周期内任务负载的时序变化，考虑时间序列输入数据与输出目标数据的映射关系，输入特征要实现多特征维度覆盖。输入数据特征为任务类型特征，具体可细分为任务综合请求到达量和任务综合类型特征向量。这些指标数据在节点资源感知子模块中同一收集处理，在算法调用时传入节点任务请求预测子模块中作为输入。

经过数据预处理后，各项指标的输入标准见表 2。

表 2. 数据特征输入标准

类别	指标	取值范围	描述	数据来源
任务特征	任务综合负载到达量 L	$L > 0 \text{ MB}$	节点执行需处理的总数据量	算网资源感知模块
	任务综合类型特征向量 Q	$Q \in [0, 1]^4$	任务类型的整体执行特性，包括任务的时间敏感性、数据需求强度、计算复杂度和引流关联性	

使用 Informer 模型进行任务负载预测。默认使用过去 12 个时间步长来预测下一时间步的节点资源性能状态。每个时间步为 5 秒钟。

- 输入格式：任务历史日志信息：二维数组 (N, n) ，其中 $N = 12$ 表示以过去 12 个时间步为预测的时间窗口， $n = 6$ 表示单个时间步所包含的数据特征维度为 6 个，模型输入数据各个维度的具体定义见表 3。

表 3. 预测模型输入维度

输入维度	各维度特征含义
$f_{t,0}$	t 时刻任务的到达时刻
$f_{t,1}$	t 时刻任务综合负载到达量
$f_{t,2} \sim f_{t,5}$	t 时刻任务综合类型特征向量

示例：[[$f_{0,0}, f_{0,1}, f_{0,2}, \dots, f_{0,n-1}$],
 $[f_{1,0}, f_{1,1}, f_{1,2}, \dots, f_{1,n-1}]$,

 $[f_{N-1,0}, f_{N-1,1}, f_{N-1,2}, \dots, f_{N-1,n-1}]$]

3.2.3.3.4. 模型输出格式

Informer 模型支持对多维度特征的并行预测，因此将输出层的数据格式定义为未来时刻节点上到达任务的总体负载量 $\hat{L}$ 和任务的综合类型特征向量 $\hat{Q}$ 。模型的输出层格式见表 4。

表 4. 数据特征输出格式

输出特征	各维度特征含义	格式
任务综合负载到达量	下一时隙节点上到达的任务所需处理的总数据量 $\hat{L}$	浮点数
任务综合类型特征向量	下一时隙节点上到达的任务整体的类型特征 $\hat{Q}$	一维向量

使用 Informer 模型对下一时刻节点上的负载到达情况与节点受损风险进行并行预测，并以多维度特征数据的格式输出。

- 输出格式：一维数组(1, n)，其中 $n=5$ 表示输出数据的特征维度为 5 个， y_0 表示下一时间步节点上到达任务的综合负载预测值， y_1 、 y_2 、 y_3 分别表示节点上到达任务的任务类型特征向量。模型输出数据各个维度的具体定义见表 5。

表 5. 预测模型输出维度

输出维度	各维度特征含义
y_0	下一时间步的任务综合负载预测值
$y_1 \sim y_4$	下一时间步的任务综合类型特征向量

示例：[$y_0, y_1, \dots, y_4$]

3.2.3.3.5. 预测算法选型

预测算法的核心目标是通过分析历史时间序列，预测未来的趋势或结果。不同的业务场景和数据特点决定了算法的选择。选型的过程需要综合考虑任务类型、数据特性以及业务需求，选择既能满足精度要求又能兼顾效率和可解释性的算法。以下将从算法原理、数据特点和适用场景等角度，介绍主流预测算法并进行性能比对。

(1) Prophet 模型

Prophet 是一种大规模时间序列预测模型，把一个时间序列看做由 3 种主要成分组成：趋势项、季节项、假期项，趋势项模拟了时间序列的非周期变化，季

节项模拟了时间序列的周期性变化，假期项模拟了假期或者其他突发事件的影响，为误差项。

Prophet 模型的核心是将时间序列表示为以下形式：

$$y(t) = g(t) + s(t) + h(t) + \epsilon_t$$

- $g(t)$: 趋势项，表示时间序列的长期变化趋势。
- $s(t)$: 季节项，表示时间序列的周期性变化。
- $h(t)$: 假期项，表示节假日或突发事件对数据的影响。
- ϵ_t : 误差项，表示随机噪声。

Prophet 能自动拟合数据中的趋势、季节和特殊事件模式，大大减少了人工干预，能处理更复杂的数据模式，但在进行非常短期的预测时，由于模型主要基于历史趋势和季节性模式进行预测，可能无法及时捕捉到近期的突发变化；而在进行非常长期的预测时，模型假设未来的趋势和季节性模式与历史相似，可能会因实际情况的变化而导致预测误差较大。

(2) LSTM 时序预测

时间序列预测是一种用于量化回归预测的方法，其中 LSTM（长短期记忆网络）被视为递归神经网络（RNN）的一种特殊形式。LSTM 模型在 RNN 的基础上引入了记忆单元和非线性门控单元，能够有效解决传统 RNN 在长时间序列中信息遗忘的问题，因此被广泛应用于时间序列问题的处理。

LSTM 的网络结构包括三个 sigmoid 层和一个 tanh 层，它们通过简单的线性操作和非线性变换相互交互。在 LSTM 时间序列分析模型中，输入层接收数据作为监督训练的历史周期数据。通过不同隐藏层之间的映射关系，利用 sigmoid 层和 tanh 层更新输出数据，从而获得未来周期的预测数据，其中包括计算资源、网络资源和能耗资源。

LSTM 的结构示意图如图 5 所示：

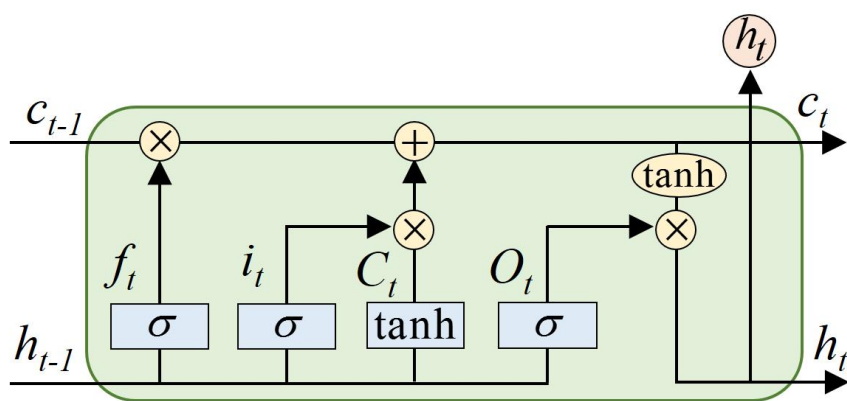


图 5. LSTM 结构示意图

- 每条黑线表示从一个节点的输出传递到其他节点的输入。
- 符号“+”表示加法运算（如矢量的加和）。
- 符号“×”表示乘法运算。
- 矩形框表示神经网络的学习单元。
- 汇合在一起的线表示向量的连接，分叉的线表示内容被复制后分发到不同的节点。

在 LSTM 的结构示意图中，每一条黑线传输着一个向量，从一个节点的输出到其他节点的输入。其中“+”号代表着运算操作(如矢量的和)，而矩形代表着学习到的神经网络层。汇合在一起的线表示向量的连接，分叉的线表示内容被复制，然后分发到不同的位置。LSTM 通过通过上述结构图，引入了记忆单元和门控机制。通过遗忘门决定需要丢弃哪些信息，通过输入门决定需要记住哪些新信息，并通过输出门生成当前时刻的输出，从而实现对时间序列的长期和短期依赖的建模。

（3）TCN 模型

TCN（Temporal Convolutional Network）即时间卷积网络，是一种基于卷积神经网络（CNN）的时间序列预测模型。其核心创新在于因果卷积与膨胀卷积的结合，解决了传统深度学习模型（如 LSTM）在并行计算和长序列处理上的瓶颈，计算效率和模型结构简单性上具有优势。相较于其他模型，TCN 模型采用显式长程依赖建模，通过膨胀卷积扩大感受野，捕捉跨时间步的依赖关系，避免梯度消失问题。同时摒弃 RNN 的时序依赖，用更清凉化的结构支持全序列并行处理，显著提升训练速度，从而支持高性能计算任务。

TCN 模型的算法原理主要包括卷积操作、残差块和预测过程。

①卷积操作：TCN 模型采用一维卷积层对时间序列数据进行处理，通过膨胀卷积扩大感受野，捕捉序列中的局部特征。膨胀卷积的数学表达式为：

$$y[t] = \sum_{k=0}^{K-1} w[k] \cdot x[t - d \cdot k]$$

其中， $y[t]$ 是卷积操作的输出， $w[k]$ 是卷积核的权重， $x[t - d \cdot k]$ 是输入序列的元素， d 是膨胀率。

②残差块：TCN 模型的残差块由两个卷积层和一个残差连接组成。残差块的引入有助于缓解深层网络中的梯度消失问题，同时提高模型的训练效率和泛化能力。残差块的数学表达式为：

$$Output = x + Conv1D(x)$$

③预测过程：TCN 模型的预测过程包括多个残差块的堆叠以及最后的输出层。整个模型的预测过程可以用以下公式表示：

$$Output = Output\ Layer(Residual\ Block(...(Residual\ Block(Input))...))$$

(5) Informer 模型

Informer 模型是一种专为处理长序列时间序列数据设计的基于 Transformer 架构的预测模型。它通过引入多种创新技术，解决了传统 Transformer 在长序列预测中的效率和性能问题。

首先 Informer 引入了概率稀疏的自注意力机制（ProbSparse Attention），这显著降低了自注意力计算的复杂度。传统的自注意力机制计算量随序列长度平方增长($O(L^2)$)，这在长序列时成本非常高。ProbSparse 通过以下公式引入稀疏化：

$$Attention(Q, K, V) = softmax(\frac{QK^T}{\sqrt{d_k}} + M)V$$

- Q, K, V ：分别表示查询、键和值矩阵。
- d_k ：键的维度，用于归一化。
- M ：掩码矩阵，用于选择贡献最大的注意力分数。

其中， M 是一个掩码矩阵，用于选择具有最高贡献的注意力评分，通过概率分布对注意力机制进行稀疏化，减少不必要的计算，仅保留最重要的注意力链接。这使得复杂度显著降低至接近线性。

Informer 还利用自注意力蒸馏技术压缩和传递多个注意力层的信息，将原本

复杂的注意力信息简化为更具表达能力的形式。这种技术通过逐层蒸馏过程，使得每一层的模型能够更专注于不同粒度的特征抽取，提高模型的整体表达能力和泛化性能。这一过程可以理解为模型在每一层中对输入信息进行“精炼”，通过子模块提取关键特征，再将这些特征与原始输入结合，最终生成更具表达能力的输出。层归一化操作则确保了信息处理的稳定性和一致性。

具体实施包括：

$$Z^l = \text{LayerNorm}(Z^{l-1} + \text{SubBlock}(Z^{l-1}))$$

- Z^{l-1} ：表示第 $l-1$ 层的输入特征。
- $\text{SubBlock}(Z^{l-1})$ ：表示第 l 层的子模块计算过程，用于提取特征并进行蒸馏处理。
- LayerNorm ：表示层归一化操作，用于稳定训练过程并加速收敛。

通过上述公式，模型在每一层中将输入特征 Z^{l-1} 与子模块提取的特征进行融合，并通过归一化操作强化层间信息的压缩和表达能力。

（6）模型优缺点分析选型

针对算网能资源预测的可靠性与资源准确确定方面，上述时间序列预测模型均有相应的优缺点，具体分析见表 6。

表 6. 时间序列分析预测方法优缺点

指标	Informer	LSTM	TCN	Prophet
长序列效率	优 (ProbSparse 注意力, 复杂度 $O(L \log L)$)	中 (门控机制, 梯度消失限制长度)	良 (膨胀卷积, 感受野指数扩展)	差 (传统分段建模, 仅适合中短序列)
多变量支持	优 (多维特征并行注意力, 联合预测)	良 (端到端学习, 需手动特征融合)	中 (多通道输入, 特征交互有限)	差 (主攻单变量, 外生变量支持弱)
实时性	优 (生成式解码, 一次性输出未来序列)	差 (逐步预测, 序列越长延迟越高)	优 (因果卷积, 并行计算低延迟)	良 (快速拟合, 但预测需重新初始化)

与 Prophet 和 LSTM 等模型相比，Informer 模型在现代战场边缘协同作战场景中具有如下优势：

①处理复杂长序列数据的能力

Informer 模型专为处理长序列时间序列数据而设计，其基于 Transformer 的

编码器-解码器结构能够高效地提取输入序列的深层特征。这种架构非常适合捕捉时间序列数据中的长期依赖关系，这对于作战场景中任务到达规模的多样性和时变性尤为重要。相比 ARIMA 和 Prophet 等传统时间序列预测模型，Informer 模型在处理非线性、非平稳和复杂动态模式的资源预测问题上表现出色。它能够处理多维特征的输入数据，挖掘并学习不同维度特征之间的关系，这对于复杂作战场景的预测性能更优越。

②捕捉长期依赖关系的能力

在作战场景下，任务到达规模不仅因任务类型不同而具有多样性，还会因作战阶段与作战区域变化而具有时变性。这种时变性导致任务负载的预测难度较大。Informer 模型通过其 ProbSparse 自注意力机制，能够紧密捕捉长期依赖关系，从而在动态变化的战场环境中表现出更高的有效性。这一能力使得 Informer 模型在预测节点资源和任务负载时能够更准确地反映实际作战需求，为作战决策提供更可靠的依据。

③实时性和计算效率

在实时作战中，对资源的要求极高，预测算法需要具备高效的计算能力和快速的响应速度。Informer 模型通过优化自注意力机制和采用生成式解码器设计，降低了计算复杂度，提高了预测效率。生成式解码器允许一次性生成整个未来序列，避免了逐步解码方式中的多次迭代计算和误差累积问题。这使得 Informer 模型在战场边缘作战中需要快速预测和调整的场景中表现出色，能够满足实时作战的需求。

因此通过对四种主流智能预测算法（Prophet、LSTM、Informer、TCN）进行综合考量与对比，最终选定 Informer 模型作为本方案的预测算法。

3.2.3.3.6. 模型原理

对于任务负载到达量的预测，本质上是一个多变量时间序列预测问题，需要一个能够高效处理时间序列数据并捕获长期依赖关系的模型，同时还需兼顾计算效率和内存使用。因此采用 Informer 模型作为智能预测算法的原型。

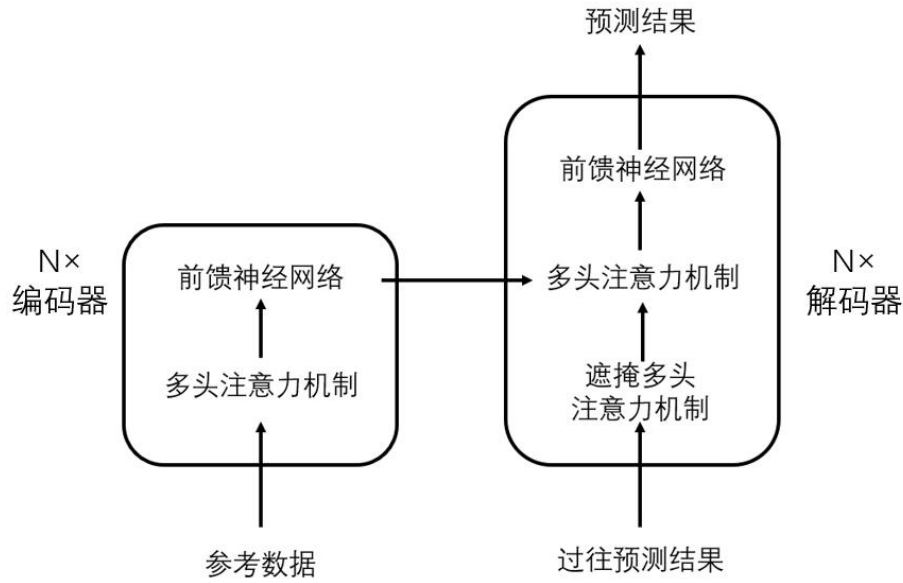


图 6. Transformer 模型的模型架构

Informer 模型的核心架构基于 Transformer 的编码器-解码器结构，具体架构如图 6 所示。这种架构非常适合捕捉时间序列数据中的长期依赖关系。在预测任务负载时，编码器的主要作用是提取输入序列（包括历史负载和协变量）的深层特征，而解码器则根据编码器提取的特征信息生成未来时隙的负载预测结果。相比传统的时间序列预测模型，Transformer 模型的优势在于其多头注意力机制能够动态地关注输入序列中不同时间点的重要性，从而有效捕捉特征间的复杂关联性。

其中编码器由多头自注意力机制和前馈神经网络构成，旨在捕捉输入序列的上下文关系及特征表示。自注意力机制通过评估输入序列中各元素的重要性，为它们分配不同的权重，从而确定不同变量间的关联性。具体而言，序列中关键元素被赋予较高权重，而次要元素则获得较低权重。前馈神经网络确保模型能够学习到输入序列中更为复杂的特征和模式。在 Transformer 预测模型中，多个编码器层叠堆砌，其中前一编码器的输出作为后一编码器的输入。这种结构的重复堆叠有助于提取输入序列的深层特征表示。

解码器由掩盖多头自注意力机制、多头自注意力机制以及前馈神经网络组成，其功能是利用编码器输出的编码矩阵和上一时间步的标签来预测当前时间步的标签。掩盖多头自注意力机制的作用在于区分输入信息中的真实值信息与先前解码器生成的预测信息，通过掩盖预测信息，防止其在参数更新过程中产生影响。

多头自注意力机制则利用当前输入与编码器处理过的编码矩阵来预测输出。与编码器类似，解码器亦由多个相同的模块堆叠而成，前一解码器的输出作为后一解码器的输入。

然而，直接使用原始的 Transformer 模型在算网任务负载预测场景中会面临一些问题。首先，随着输入序列长度和特征维度的增加，Transformer 的全注意力机制会导致计算复杂度和内存占用急剧增加，这在任务负载预测中是不可接受的。其次，Transformer 模型的解码器采用逐步预测（step-by-step）的方式，每次预测一个时间步长，再将结果反馈到模型中进行下一步预测，这种方式效率较低且容易产生误差累积。为了解决这些问题，本算法采用了 Informer 模型，并对其架构进行了优化以更好地适配负载预测任务。

（1）稀疏自注意力机制

随着输入参数的增多，Transformer 模型的注意力机制往往需要比较大量的点积对，导致计算时间复杂度与内存占用过高。为了适应场景需求，减少资源占用问题，本算法使用的 Informer 模型引入稀疏的自注意力机制。在实际的算法运行过程中，只有少数的点积对在主要的注意力函数中存在贡献，其他的点积对可以减少相应的判断权重，这些点积对可以被忽略。

对于如何识别出可忽略的权重，算法使用了 KL 散度公式，将第 s 个查询向量 Q 的注意力权重看作一个概率分布 $p(k_j; q_s)$ ，当一些值对注意力权重产生显著的影响时就会使 p 偏离均匀分布，当这些值对注意力的权重接近于均匀分布时，这些值就成了冗余值。基于权重的注意力概率分布计算公式如下：

$$p(k_j; q_s) = \frac{k(q_s, k_j)}{\sum_{j=1}^{L_K} k(q_s, k_j)}$$

其中 $k(q_i, k_j)$ 表示查询向量 q_s 和键 k_j 的相似性度量函数，通常采用非对称指数核函数 $\exp(\frac{q_i k_j^T}{\sqrt{d}})$ ， L_K 为 Q 的长度，分母表示对所有键向量的核函数值求和。

将上式代入 KL 散度公式中即可得到稀疏性度量，计算公式如下：

$$M(q_s, K) = \ln \frac{1}{L_K} \sum_{j=1}^{L_K} \exp(\frac{q_s k_j^T}{\sqrt{d}}) - \frac{1}{L_K} \sum_{j=1}^{L_K} \frac{q_s k_j^T}{\sqrt{d}}$$

如果第 s 个 Q 的 $M(q_s, K)$ 取值较大，就表明它的注意力权重更加多样化，可能会包含点积对中影响力较高的一部分。因此对于所有的 Q 来说，可以选取 $M(q_s, K)$

排名最靠前的若干个 Q 作为 softmax 函数的输入矩阵，选取的个数 $u = c \cdot \ln L_Q$ 。 L_Q 表示 Q 矩阵的时间步长， c 是一个常数采样因子。那么稀疏注意力机制的运算过程就可以表示为

$$A(Q, K, V) = \text{softmax}\left(\frac{\hat{Q}k_j^T}{\sqrt{d}}\right)V$$

其中 Q 表示查询向量； K 表示键值； V 表示数据的相似度权重。

输入的数据通过使用上述优化方法，在计算每个 K 值时，只需要计算 $O(\ln L_Q)$ 次点积，并且内存的使用量也下降为 $O(L_K \cdot \ln L_Q)$ 。

（2）生成式解码器设计

在解码器部分，Informer 模型摒弃了传统 Transformer 的逐步解码方式，改用生成式解码器，一次性生成整个预测序列。这种设计避免了多次迭代计算，大幅提升了运行速度，同时减少了误差的累积。生成式解码器由一个带掩码的稀疏注意力模型和一个多头注意力模型组成，解码器的输入由两部分组成：第一部分输入为编译器的输出，包含数据的 K 值与 V 值；第二部分输入是由上一个解码器的输出用 0 遮挡后得到的向量。通过稀疏多头注意力机制，解码器能够快速生成所有时间步的预测结果，而无需逐步迭代。对于任务负载预测来说，这种解码方式更加高效，也更符合实际场景中对快速预测的需求。流程如图 7 所示。

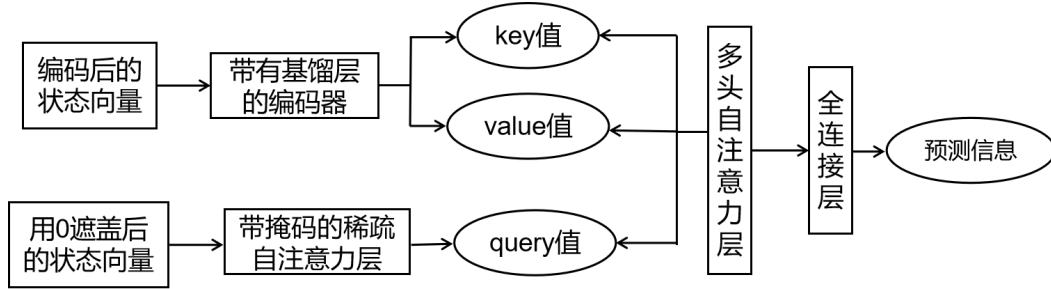


图 7. Informer 模型的解码器运行流程

为了实现多特征的并行预测，本方案中将输出层设定为多个独立的全连接层同时获得任务到达量，任务所需资源的预测值。全连接层的数量等于所需预测的特征的维度。

3. 2. 4. 任务需求映射子模块

3. 2. 4. 1. 模块功能

从任务请求的资源分配优化角度，在战场环境下，处理不同类型的任务对资

源的需求往往具有很大的差异，一些任务计算复杂度高，可能需要占用大量的计算资源，但对网络带宽的需求较少，另一些任务则可能需要较高的网络带宽来传输数据。因此需要对不同类型任务的资源需求进行细粒度感知。从面向节点资源负载变化趋势角度，负载预测模块虽然能预测未来时刻的任务的总负载到达量，但仅依赖任务负载到达量，不足以为系统的精准调度提供依据，因为任务到达量虽然可以反映任务处理数据量的变化，但它并不能揭示节点未来需要为任务执行提供的算力和网络资源。

为了确保系统能够灵活、合理地调度资源，设计了任务需求映射子模块以提供细粒度的任务资源需求感知能力。因此在任务需求映射模块中，采用神经网络实现资源的拟合度量，该算法可以有效捕捉任务类型与资源消耗之间的复杂非线性关系，灵活地处理任务类型的多样性。同时应对战场环境中的动态性和负载，适应系统负载的波动，为系统的智能决策提供支撑，从而使得系统实现更精确的任务调度和资源分配。

3.2.4.2. 模块设计思路

以 LightGBM 与深度神经网络为核心的任务资源需求映射方案包括特征输入、LightGBM 自动分组与标签提取、特征拼接与深度神经网络拟合，这四个步骤。

(1) 特征输入

在算网资源感知子模块中已实现面向任务执行需求的感知，即将每个任务的原始属性抽象化为一个结构化的任务类型特征向量，在算网资源映射模块中将使用这个任务类型特征向量作为拟合算法的输入。

(2) LightGBM 自动分组与标签提取

LightGBM 是一种高效的集成决策树模型，可以理解为很多棵决策树的组合体。每一棵树独立地根据输入的特征向量对任务展开决策与归纳。树上的每一次分支，就是根据某个任务特征，比如任务的时延敏感度、数据需求强度等，来判断要走哪一条路，最终把任务分到某一个叶子节点。这个决策的阈值不是人工主观设定的，而是 LightGBM 在训练过程中学习到的。

LightGBM 的训练过程，就是反复用历史任务样本，即带有真实标签的任务数据自动地“分区、打标签”。它会不断学习改进对于特征分裂点和分裂顺序的选

择。分裂点的本质就是找到当前数据集在什么特征、什么取值处划分，能使信息增益最大化。每一个特征的候选切分点，都会被计算切分前后总的残差平方和的减少量。这个减少量就对应着信息增益。减少量最大，信息增益就最大，这个候选切分点就被定为一个分裂点。而分裂顺序的本质就是每一层优先根据当前最能增进模型纯度的特征进行分裂，直到节点中的样本非常相似或数量很少。增加纯度是指每一次分裂，希望把一类样本尽量聚合在一起，让每个叶子节点内的样本在目标值上趋于一致。决策树在选择分裂点时，总是优先选择那个可以让分裂后每个新分组内部尽量一致、差异最小的特征和阈值。这样叶子节点最终的预测才会更准确。总之 LightGBM 在训练过程中，会不断优化分裂点与分裂顺序的选择使得最终组内数据更一致，模型拟合误差最小化。LightGBM 的训练原理如图 8 所示：

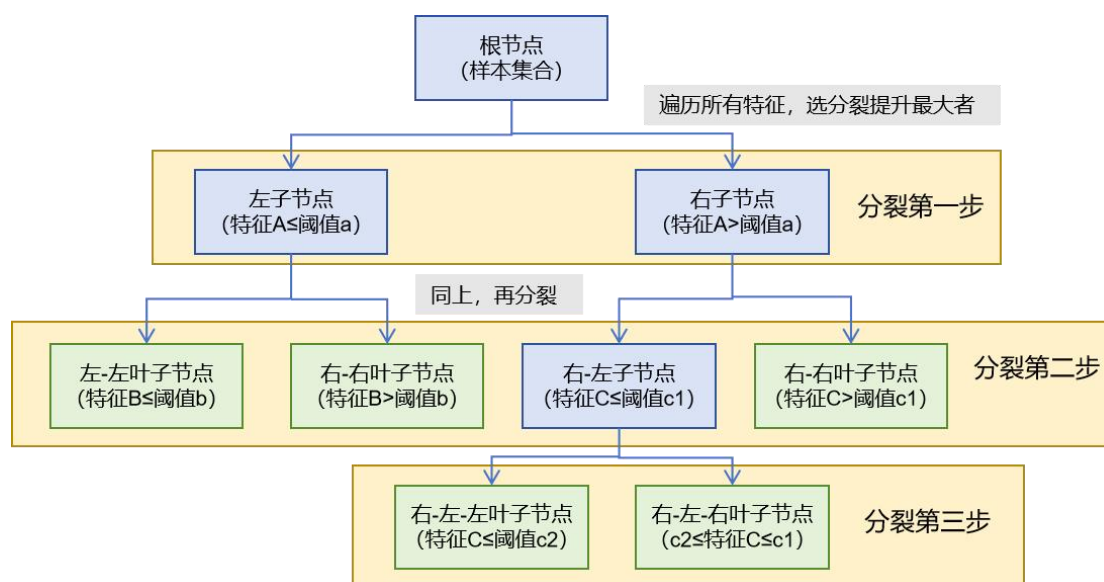


图 8. LightGBM 训练原理示意图

在算法应用时，LightGBM 通过一系列的规则决策，最终把每个输入的任务划分到某一个叶子节点。每个叶子节点本质上代表了一种任务特征的组合，比如“低时延容忍+高数据需求+低计算复杂度”这个组合可能被映射到某一个特别的叶子节点。这些叶子节点是有限的，因此任务被分配到哪些叶子节点，就相当于给这个任务贴上了“属于哪几类子群体”的代表性标签，如图 9 所示：

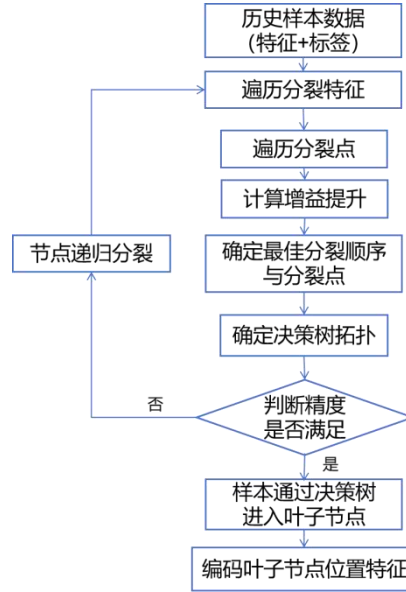


图 9. 基于 LightGBM 的任务特征拓展

假设有 100 棵树，这个任务在每棵树上都会有一个叶子节点编号，比如树 1 的第 3 个叶子，树 2 的第 12 个叶子，树 3 的第 5 个叶子……。这些叶子节点编号集合可以理解为这是一套经过 LightGBM 智能归纳之后，代表当前任务的描述标签。然后把这个所有叶子节点编号的集合通过 `embedding` 的形式转换成一个新的稀疏向量。LightGBM 已经用过往数据经验，把原始复杂的任务特征转化成一组能更好描述任务空间区块的标签，这些叶子节点编号反映了模型基于历史数据对任务本身特征的高度浓缩和分组，可以更加细致准确地描述任务类型特征。

(3) 特征拼接

因为传统 LightGBM 只支持单变量的回归，而对于任务资源需求的拟合需要同时考虑任务的算力资源需求与网络资源需求，因此采用了多目标任务的单目标拆分建模策略，即针对每个资源类型（如算力、网络带宽），分别建立并训练独立的 LightGBM 模型。每个模型利用历史数据，专门归纳和分组其目标资源的规律，且每个目标变量形成一套叶子节点标签特征。

为了最大化使用所有信息，采用“特征融合”方式，将不同映射指标对应的 LightGBM 模型产生的新特征与原始任务特征向量拼接，形成一个联合特征向量，这样综合利用了原始属性信息和分组后遗传的经验数据。

(4) 深度神经网络（DNN）资源需求预测

上述联合特征向量被输入到深度神经网络（DNN）。DNN 拥有强大的模式学习和复杂关系建模能力，能够从拼接后的信息中学习和分析多种特征间交互及其对算力和带宽需求的影响。经过不断学习和优化，DNN 最终能准确输出每个任务的所需的算力配额和所需的网络带宽配额。

任务资源需求映射模块的总体流程，如图 10 所示：

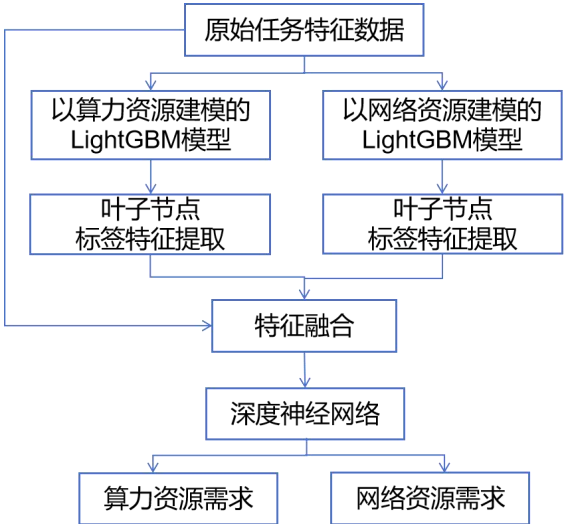


图 10. 任务资源需求映射模块总体流程

3. 2. 4. 3. 任务-算力资源的映射模型

3. 2. 4. 3. 1. 任务-资源映射框架

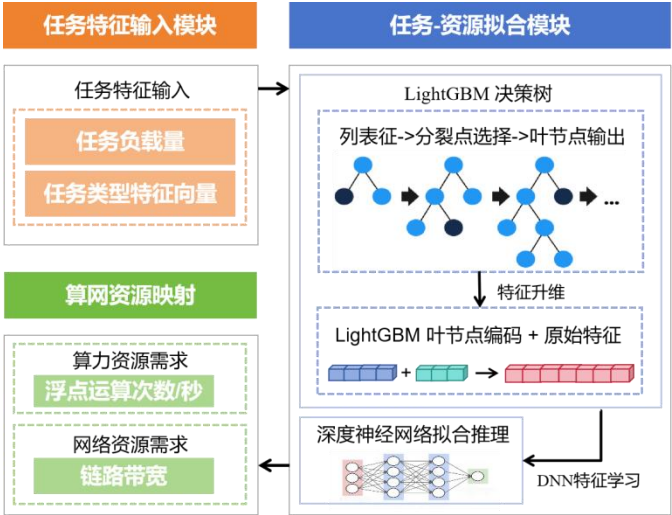


图 11. 作战场景中的任务-资源映射框架

为实现作战环境中对未来任务的精准预测，需准确解析作战任务请求对算网资源的需求，构建作战“任务-算力资源”映射。为此需综合考虑任务多重特征属性，解析作战任务对算力资源、网络资源的多样化需求，从而进一步构建任务

与资源的映射关系。该映射关系的构建包含任务特征输入、任务-资源拟合以及算网资源表征三个部分，其框架如图 11 所示。

(1) 任务特征输入

任务特征输入是指为了有效拟合任务与其资源需求之间的隐式映射关系，从不同维度对任务特征进行细粒度的分析来反映任务对资源的差异化需求和服务倾向，需要对任务特征建模作为算法的输入。任务特征模型需精确描述任务对于服务执行性能的需求，以帮助系统更精确地学习不同类型任务对计算资源、网络资源的具体需求，从而实现资源的优化分配和高效利用，确保任务能够按照预期的性能和效率执行。

(2) 任务-资源拟合

任务-资源拟合是指将对输入的任务信息进行特征提取与学习，通过深度神经网络，采用“数据驱动”的方式，拟合出任务特征与算网资源之间的映射关系，从而实现对于任务资源需求的感知。

(3) 算网资源表征

算网资源表征是指基于模型的拟合结果，对输入任务的算网资源需求进行标准化描述，从而为任务调度和资源分配提供统一的量化指标。算网资源表征主要包括算力资源与网络资源这两个方面的表征，本算法中将算网任务的算力需求转化为算力规模需求，将算网任务的网络需求转化为网络资源带宽需求。

3.2.4.3.2. 算网资源需求映射

通过任务与资源关系的拟合，实现对任务计算和网络资源规模需求的映射。映射模型输出算网资源需求预测向量，包括两项关键指标：算力资源需求规模和网络资源需求规模。算力需求通过 FLOPS（每秒浮点运算次数）表示计算能力需求，反映任务执行过程中每秒需要完成的浮点计算次数，以此评估计算复杂度。输入特征包含任务的特征参数，这些特征经拟合算法得出 FLOPS 需求。网络资源需求则通过 Mbps（兆比特每秒）量化带宽需求，表示单位时间内的数据传输容量需求。这一度量反映了任务在执行过程中需要的数据传输速率。具体模型输出数据格式设置见表 7。

表 7. 模型训练输出数据格式

输出特征	各维度特征含义	格式
------	---------	----

算力性能需求 (FLOPS)	任务执行过程中每秒需完成的浮点运算次数 (FLOPS)，用于表征任务算力资源需求。	浮点数
网络带宽 (Mbps)	任务执行过程中所需的数据传输速率,单位为兆比特每秒 (Mbps)，用于表征任务网络资源需求。	浮点数

3.2.4.3.3. 任务特征模型构建

算力任务类型与任务的资源需求密切相关，比如计算复杂度高的任务在执行过程中需要占用较多的计算资源（如 CPU 或 GPU 计算能力）。这类任务通常对强大的处理能力和高速计算有很高的需求，涉及大量的数学运算或复杂的算法。为了完成这些大量的算术计算和数据处理，通常需要高性能的处理器，如多核 CPU 或 GPU，这往往导致较长的执行时间。尽管这些任务处理大量数据，其主要瓶颈在于计算能力，而非数据存取，因此对内存或 I/O 带宽的需求相对较低。在作战场景中，计算密集型任务包括：目标识别与分类、作战模拟与推演、路径规划与导航、物理仿真与动态建模等等。

而时延敏感性较高任务通常需要快速响应，任务完成的时机直接影响其效果或成败。它们依赖于极低的通信延迟和计算响应时间，以确保操作的即时性和连续性。时延敏感型任务的性能往往受限于网络延迟、计算处理时间以及系统整体的实时性。在作战场景中，时延敏感型任务包括：导弹制导与防空拦截、实时战术指挥、动态路径规划、近距离通信协同等等。

作战任务表现出特征的多样化和资源需求的异构化，不同的算力任务具有不同的计算特性、数据处理需求以及资源配置要求。因此在构建任务特征模型时，需考虑任务请求在计算复杂度，时延敏感度等方面的特征。为了更全面地描述每种类型的任务执行特征，将任务负载量与任务类型特征向量共同组成任务特征，作为映射模型的输入：

$$Task = \{L, Q\} = \{L, T, D, C, R\}$$

其中 L 表示任务的负载量， Q 表示任务类型特征向量。 T, D, C, R 是任务类型特征向量具体的三个维度。 T 表示任务的时间敏感性， D 表示数据需求强度， C 代表计算复杂度， R 代表引流关联性。 T, D, C, R 是任务类型特征向量的三个维度的具体定义同前节一致，分别可通过对任务的最大时延容忍，执行单个任务需要传输的数据量和执行单个任务需要处理的数据量归一化得到。

任务-资源映射模型的作用是将输入的任务负载信息映射为算网资源需求信息，在实际应用算法时，根据算法时间驱动以时间驱动的两种触发模式，任务需求映射子模块以算网资源感知子模块感知所得的输入请求的任务类型特征向量或是节点任务请求预测子模块输出的未来任务综合负载量和任务综合类型特征向量作为模型输入。具体模型训练输入数据格式设置见表 8。

表 8 模型训练输入数据格式

输入特征	各维度特征含义	取值范围
任务的负载量	任务执行时需要处理的数据量	$L > 0$
时间敏感性	描述任务对完成时间的严格要求程度，可由任务的 最大容忍时延归一化得到	$T \in (0,1)$
数据需求强度	描述任务在执行过程中需要传输的数据量大小， 可由执行单个任务所需传输的数据量归一化得到	$D \in (0,1)$
计算复杂度	描述任务在执行过程中对计算资源的需求程度， 可由执行单个任务所需处理的数据量归一化得到	$C \in (0,1)$
引流关联性	描述具有依赖性的任务对于后续任务流量的影响。 可由执行单个任务依概率触发后续任务的数据量归一化得到	$R \in (0,1)$

3. 2. 4. 3. 4. 任务与资源需求关系拟合

为了挖掘任务执行性能与资源需求之间的映射关系，将 LightGBM 算法与神经网络结合作为任务与资源关系拟合算法，这种模型设计的优势在于这种组合能够最大化利用两者的优点，从而提升模型的整体性能。LightGBM 通过高效的梯度提升决策树算法处理结构化数据，能够快速训练并生成准确的预测结果，同时提供任务特征重要性评估，这对于特征选择和理解模型决策至关重要。另一方面，神经网络在捕捉复杂的非线性关系和处理大规模数据方面具有强大的能力，能够从数据中自动学习深层次的模式和特征。当这两种方法结合时，LightGBM 可以用来处理数据的基本结构和特征，而神经网络则进一步挖掘更复杂的模式和关系，从而在实际应用中显著提高预测的准确性和模型的泛化能力。

LightGBM 与深度神经网络协同的整体思想是先通过树模型提取叶节点路径特征，将其编码为高维稀疏向量，再输入到神经网络中作为额外特征。这一方式既能够利用树模型的局部拟合能力，又能通过 DNN 的特征学习进一步提高分类性能。

基于 LightGBM 树型模型结合方法，本度量模型使用神经网络与 LightGBM 树型模型结合的方法构建度量拟合算法，构建算法的具体度量流程如下：

首先对输入数据进行标准化和独热编码等预处理，通过 LightGBM 树模型对相关输入数据进行第一步预测后将预测结果输入神经网络中进行进一步学习，LightGBM 算法作为高效的梯度决策树算法，可以通过加权不同特征的分裂次数，统计每个特征对叶子节点输出的贡献，从而分析出不同特征对于预测结果的贡献大小，并利用分裂特征空间的方式，将样本分配到有限数量的叶子节点中。对于每个样本，记录其在每棵树中最终所在的叶子节点的索引。将叶子节点索引结合叶子节点的输出值（如预测值或概率）合并作为新的特征输入到深度神经网络中。新生成的特征可以捕捉到样本在特征空间中的局部信息使得神经网络在此基础上进一步学习特征之间的深层次关系，从而提升模型的预测能力。

然后输入神经网络的数据经过三层卷积运算后得到神经网络输出，这是训练的第二步。神经网络模型使用 Sequential 模型，构建一个全连接神经网络，全连接层包含 3 个全连接神经网络，定义损失函数为均方误差，使用默认激活函数。

神经网络训练完成后，此时判断神经网络的数据迭代次数，若还未完成 50 次的迭代学习，则进行第三步训练：网络训练。通过与原始数据输入进行网络训练，指定优化器为 adam，每批次样本数定义为 32，学习率设置为 20%。

训练完成后送回神经网络继续进行卷积运算，对神经网络输出进行纠正，完成 50 次迭代，这是度量模型的第四步训练。

（1）LightGBM 算法

针对步骤一中的 LightGBM 算法初级进行解析，基于 LightGBM 的任务度量算法包括数据特征选取量化、决策树优化过程，通过 LightGBM 树模型对任务输入数据进行初步预测后，将预测处理的数据输入神经网络中进行深度拟合学习。

①LightGBM 算法原理

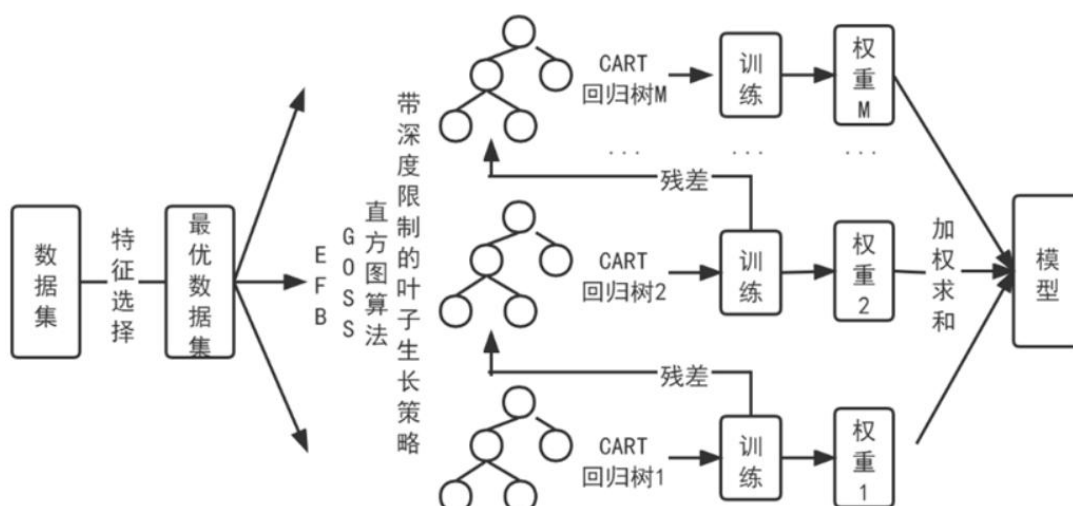


图 12. LightGBM 算法流程图

LightGBM 是一个性能高度优化的 GBDT（梯度提升决策树）算法，LightGBM 的算法流程，如图 12 所示。为进一步说明算法中各部分的优化原理，可以将 LightGBM 的优化用公式表达，并分别介绍各部分算法在 LightGBM 中的应用：

$$LightGBM = XGBoost + Histogram + GOSS + EFB$$

其中 XGBoost 是 LightGBM 的基础框架，Histogram 是引入的直方图优化算法，可以加速特征分裂点的查找。GOSS 是基于梯度的单边采样方法，可以减少数据量，提高训练速度。EFB 代表互斥特征捆绑，用于减少特征维度，提高内存利用率。

GBDT 的核心思想是通过多轮迭代逐步优化模型，每一轮拟合前一轮的残差，逐步逼近真实值。残差是当前预测值与真实值之间的误差，GBDT 通过计算损失函数对预测值的梯度，确定优化方向，并生成一棵新的决策树来修正预测结果。

每棵决策树通过选择特征分裂点最大化对残差的拟合效果，最终所有树的输出结果加总，形成最终的预测值。通过这种逐步优化的方式，GBDT 能够高效地提升模型性能，适用于回归和分类等任务。

而 XGBoost 算法作为 GBDT 的一种改进，在训练时是基于一种预排序的思想来寻找特征中的最佳分割点，其主要原理思路是通过每次拟合上次的残差（上次实际值与目标值之差），从而每次生成一棵树（CART 回归树），最终将所有的树加起来得到最终目标。拟合原理函数包括模型预测、模型推导过程，优化函数包括：

$$\hat{y}_i = \sum_{t=1}^T f_t(x_i), f_t \in F$$

$$L = \sum_{i=1}^n l(y_i, \hat{y}_i)$$

其中 x_i 为第 i 个样本的输入数据，即数据集中第 i 个任务的负载量与任务类型特征向量； T 代表特征树的数量， $\sum_{i=1}^n l(y_i, \hat{y}_i)$ 表示对 n 个样本的损失值求和； $l(y_i, \hat{y}_i)$ 表示与对于算网资源的预测值 $\hat{y}_i$ 及任务实际分配到的真实值 y_i 的损失函数（该函数中作为标签的实际算网资源 y_i 是已知的，但是预测值 $\hat{y}_i$ 随着树模型的不同而在变换，所以变量是 $\hat{y}_i$ ）。

为降低数据分布对模型准确性的影响，LightGBM 提出了基于梯度的单边采样（GOSS）。GOSS 保留梯度大的所有实例，对梯度小的实例进行随机抽样。为补偿对数据分布的影响，将更多的注意力放在训练不足的实例上，而不改变太多的原始数据分布，在 GOSS 方法中，首先根据梯度绝对值对训练实例进行降序排序；其次保留梯度较大的前 $a \times 100\%$ 的实例，从而形成实例子集 A ；接着，对于由剩余 $(1-a) \times 100\%$ 梯度较小的实例构成的集合 A_c ，再随机选择一个大小为 $b \times |A_c|$ 的子集 B ；根据在子集 $A \cup B$ 上估计的方差增益 $\tilde{V}_j(d)$ 来分割实例，即：

$$\tilde{V}_j(d) = \frac{1}{n} \left(\frac{\left(\sum_{x_i \in A_l} g_i + \frac{1-a}{b} \sum_{x_i \in B_l} g_i \right)^2}{n_l^j(d)} + \frac{\left(\sum_{x_i \in A_r} g_i + \frac{1-a}{b} \sum_{x_i \in B_r} g_i \right)^2}{n_r^j(d)} \right)$$

式中：

- $\tilde{V}_j(d)$ 表示特征 j 在分裂点 d 处的分裂增益(即分裂后模型性能的提升)，这是 GOSS 方法中用于衡量分裂效果的指标。
- n 表示训练样本的总数。
- $A_l = \{x_i \in A: x_{ij} \leq d\}$ 表示集合 A 中特征 j 的值小于等于分裂点 d 的样本集合。
- $A_r = \{x_i \in A: x_{ij} > d\}$ 表示集合 A 中特征 j 的值大于分裂点 d 的样本集合。
- $B_l = \{x_i \in B: x_{ij} \leq d\}$ 表示集合 B 中特征 j 的值小于等于分裂点 d 的样本集合。
- $B_r = \{x_i \in B: x_{ij} > d\}$ 表示集合 B 中特征 j 的值大于分裂点 d 的样本集合。

- g_i 表示样本 x_i 的梯度值，反映当前样本对损失函数的贡献大小。
- α 表示 GOSS 方法中保留高梯度样本的比例
- b 表示 GOSS 方法中随机采样的低梯度样本的比例。
- $n_l^j(d) = |A_l| + \frac{1-\alpha}{b}|B_l|$ 表示特征 j 在分裂点 d 左侧样本的有效数量，包含高梯度样本 A_l 和经过缩放的低梯度样本 B_l 。
- $n_r^j(d) = |A_r| + \frac{1-\alpha}{b}|B_r|$ 表示特征 j 在分裂点 d 右侧样本的有效数量，包含高梯度样本 A_r 和经过缩放的低梯度样本 B_r 。

为了避免了特征值为零的不必要计算，LightGBM 算法通过引入互斥特征捆绑（EFB），从特征包构建与从单个特征构建相同的特征直方图，有效降低特征提取复杂度。

基于 LightGBM 算法的原理，将 LightGBM 算法与 Sequential 神经网络进行结合，综合发挥两者的优势，充分利用 LightGBM 在处理数据基本结构和特征上的能力，同时借助 Sequential 神经网络对复杂关系的建模能力，构建一阶段任务-算网资源度量模型。

②数据特征选取量化

分析算网任务执行特征与资源需求特征，输入样本数据集为 $\{((x_{1,n}, \dots, x_{j,n}), y_n)\}_{n=1}^N$ ，其中 $(x_{1,n}, \dots, x_{j,n})$ 表示第 n 个样本的任务负载量与任务类型特征向量构成的数组， $x_{j,n}$ 表示第 n 个样本的第 j 维特征向量。 y_n 是对应的真实标签，即实际占用的算网资源。

LightGBM 模型的预测输出数据 $\hat{y}_n$ 是通过多个基学习器（决策树）累加得到的。假设模型由 K 棵决策树 $f_k(x)$ 组成，则对于任务特征样本 $x_{j,n}$ 的预测输出公式为：

$$\hat{y}_n = F(x_{j,n}) = \sum_{k=1}^K \sum_{j=1}^J f_k(x_{j,n})$$

其中 $F(x_{j,n})$ 是 LightGBM 模型的总预测激活函数。 K 表示 LightGBM 模型中基学习器（决策树）的总数， J 表示每棵决策树中的叶节点总数。 $f_k(x_{j,n})$ 表示第 k 棵决策树在样本 $x_{j,n}$ 上的输出。

每一棵决策树 $f_k(x)$ 可以被看作是一组分段常数函数。对于给定的特征 x ，决策树根据预先定义的分裂规则将输入特征划分到不同的叶节点中，每个叶节点都

有一个固定的预测值 v_j 。

假设第 k 棵树有 J_k 个叶节点，每个节点的输出为 v_j^k ，那么该树对输入 x 的输出可以表示为：

$$f_k(x) = v_j^k \text{ if } x \in \text{Region } R_j^k$$

其中 $f_k(x)$ 表示第 k 棵决策树对输入样本 x 的预测值。 $\text{Region } R_j^k$ 表示第 k 棵树的第 j 个叶节点对应的特征空间区域。 v_j^k 表示该区域对应的固定预测值。

③ 决策树优化

LightGBM 是通过梯度提升的方式来逐步构建优化每一棵树。在第 k 轮迭代中，LightGBM 通过拟合损失函数对当前残差的负梯度来生成新的树。假设损失函数为 $L(y, \hat{y})$ ，则模型在第 k 轮的目标是最小化以下损失：

$$\mathcal{L}^k = \sum_{i=1}^N L(y_i, F^{k-1}(x_i) + f_k(x_i))$$

其中 $F^{k-1}(x_i)$ 是前 $k-1$ 棵树的累加预测结果。 $f_k(x_i)$ 是第 k 棵树的输出。

故总结 LightGBM 模型的输入输出过程，最终可以用如下公式表示：

$$\hat{y}_i = F(x_i) = F^0(x_i) + \sum_{k=1}^K f_k(x_i)$$

在这里，每个 $f_k(x_i)$ 表示通过拟合当前损失函数负梯度生成的第 k 棵决策树的输出。整个模型通过累加每一轮的基学习器的预测值，逐步逼近真实值 y_i 。

(2) Sequential 神经网络模型

在数据驱动度量拟合算法中 Sequential 神经网络模型负责对输入层处理的数据进行资源预测与网络训练，其全连接层模型架构如图 13 所示。

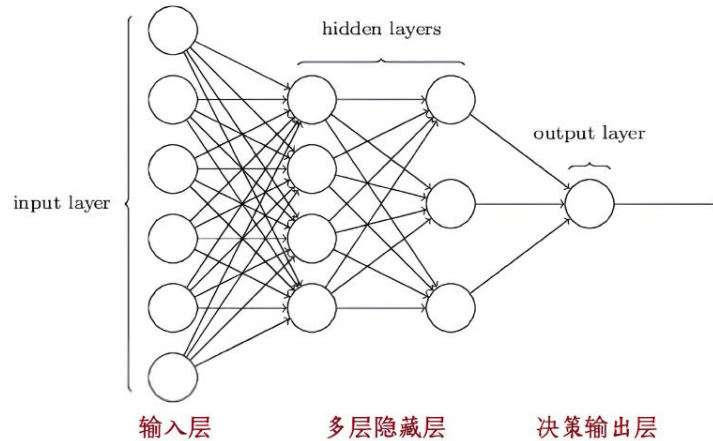


图 13. 全连接层模型架构

拟合网络输入层：输入层是拟合度量神经网络的第一层，它的主要功能是接收原始数据输入并将其传递到网络的下一层，输入层负责对预处理后的算网任务特征参数进行量化，

拟合网络隐藏层：隐藏层为拟合度量神经网络的中间部分，负责提取输入层算网任务的高级特征，并进行拟合的非线性变换，以便获得算网任务特征与算网资源之间的映射关系，拟合映射关系主要通过网络权重矩阵 U 与激活函数 F 构建，权重矩阵 U 负责对输入层输入的算网任务特征进行权重计算划分，激活函数 F 则对权重矩阵处理后的数据进行网络计算。

拟合网络输出层：输出层负责对隐藏层处理算网任务的最终特征表示转换成算力资源和网络资源需求。

基于 Sequential 神经网络，针对 LightGBM 预测处理的服务参数进行输入处理，获得多元算网任务所需的算网资源需求。分析神经网络拟合模型，具体度量流程包括任务特征表征，度量激活函数设计、网络前向传播设计与度量输出损失函数模块，具体流程如下：

任务特征量表征：任务特征量表征的目的在于将原始数据转换成适合神经网络处理的格式，这包括归一化或标准化数据以消除不同量纲的影响，处理缺失值以确保数据完整性，编码类别特征以转换非数值数据，以及可能的特征工程来提取有用的信息，从而提高模型的训练效率、准确性和泛化能力。 $Ax(x_1, x_2, \dots, x_j, x_n)$ 为一个包含 n 个特征的输入向量分别为算网任务的输入特征数据。考虑不同任务类型在输入层的表征，其输入特征集合表示为：

$$X = \{x_1, x_2, \dots, x_j, x_n\}$$

$$z_i = w_1x_1 + \dots + w_jx_j + b$$

其中 x_j 为第 j 个输入特征，表示具体算力任务的服务参数，以计算密集型任务下图像识别为例 x_j 可表示为卷积神经网络模型的、学习率、训练总迭代轮数、训练停止的精度阈值。 z 是不同任务输入特征加权求和的结果， w_i 是第 i 个输入特征的权重因子， b 是偏置项。

度量激活函数设计：激活函数设计是为了引入非线性因素到神经网络模型中，使得网络能够学习和拟合复杂的数据关系和模式，帮助神经网络在各种任务中取得更好的性能。特征量化的结果通常会通过一个非线性激活函数 F 进行转换，以引入非线性特性，从而拟合任务能够与算网资源需求之间的关系。采用双曲正切

函数作为激活函数 F ，计算公式为：

$$F(z) = \tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

网络前向传播设计：网络反向传播是指在神经网络训练过程中，通过计算损失函数关于网络参数的梯度来更新网络的权重和偏置，以此最小化损失函数，提高模型预测的准确性。对于单隐藏层，神经元 i 的输出结果可表示为：

$$s_i = F(z_i)$$

对于多层的隐藏层设计，需考虑不同神经元与网络层数的影响，因此需要对不同神经元的隐藏层计算特征进行前向传播计算，前向传播的计算可以表示为：

$$s_j^l = F^l(\sum u_{j,i}^{ll} * s_i^{l-1} + b_j^l)$$

其中 s_j^l 是表示第 l 层网络中第 j 个神经元的输出值， $u_{j,i}^{ll}$ 是表示第 l 层中第 j 个神经元与第 $l-1$ 层中第 k 个神经元之间的权重，其决定了上一层神经元 s_k^{l-1} 对当前神经元 s_j^l 的影响大小。 b_j^l 是第 l 层第 j 个神经元的偏置。 F^l 为激活函数，负责对加权求和的任务特征进行非线性变化，引入非线性关系。

度量输出损失函数：损失函数通过量化模型预测与真实值之间的差异来衡量模型的性能，为网络训练提供反馈信号，引导权重和偏置的调整方向，确保模型能够通过最小化损失，从而提高预测的准确性和泛化能力。输出层负责对隐藏层处理算网任务的最终特征表示转换成算力资源和网络资源需求，输出层计算公式为：

$$o_j = G^l(v_j * s_j^l + c^l)$$

其中 G^l 为第 l 层的激活函数， v_j 是第 l 层网络中第 j 个神经元的输出值 s_j^l 到输出层 y_j 的权重矩阵， c^l 是第 l 层的偏差向量。

在训练过程中，神经网络的输出与真实值之间的差异通过损失函数来衡量，并通过损失函数优化拟合度量效果，定义损失函数 $Loss$ 为：

$$Loss = MSE = \frac{1}{N} \sum_{j=1}^N (o_j - \hat{o}_j)^2$$

其中模型选用均方误差（MSE），即预测值与真实值之间误差的平方平均值，作为损失函数 $Loss$ 。 N 表示样本总数， $\hat{o}_j$ 为表示拟合感知的任务算网资源需求向量，包含关于不同资源指标的需求预测值， o_j 表示实际测试集的任务分配到的算网资

源规模向量，包含任务实际占用的不同资源指标。

基于多层神经网络模型与 LightGBM 算法结合的方式，可以精确解析算网任务请求，获得多元任务场景、任务类型与具体应用下的算网资源需求映射关系。

3.2.5. 算法整体输入输出

3.2.5.1. 算法部署阶段的输入输出

(1) 时间驱动模式：

当算法实际部署时，每隔固定的时隙（预测周期）进行一次算法调用。算法整体的输入如下表 9 所示，其中 f_{task_i} 表示历史任务序列中关于第 i 个任务的数据记录，对于每个到达任务的数据维度为 5。历史序列的长度为预测算法调用前 12 时隙内到达的所有任务记录，即如果前 12 时隙内总共到达了 N 个任务，则历史任务序列应记录了 N 个任务的特征数据。

表 9. 算法整体输入

输入维度	各维度特征含义	类型	字节数	值域	单位
$f_{task_{i,0}}$	第 i 个任务的到达时刻	ISO 8601	—	合法日期时间	YYYY-MM-DDTHH:MM:SS 如 2021-12-31T23:59:59
$f_{task_{i,1}}$	第 i 个任务的最大时延容忍值	Double	8	$(0, 10^4)$	s
$f_{task_{i,2}}$	第 i 个任务的传输数据量	Double	8	$(0, 10^4)$	MB
$f_{task_{i,3}}$	第 i 个任务的处理数据量	Double	8	$(0, 10^8)$	MB
$f_{task_{i,4}}$	此到达时刻对应的作战阶段	String	—	{"战备值班" "作战准备", "火力打击", "毁伤评估"}	—

示例：[[$f_{task_{0,0}}, f_{task_{0,1}}, f_{task_{0,2}}, \dots, f_{task_{0,4}}$],
 $[f_{task_{1,0}}, f_{task_{1,1}}, f_{task_{1,2}}, \dots, f_{task_{1,4}}]$,

 $[f_{task_{N,0}}, f_{task_{N,1}}, f_{task_{N,2}}, \dots, f_{task_{N,4}}]$]

算法整体的输出如下表 10 所示：

表 10. 预测模型输出维度

输出维度	各维度特征含义	类型	字节数	值域	单位
------	---------	----	-----	----	----

y_0	下一时隙节点上算力性能总需求	Double	8	$(0, 10^8)$	FLOPS
y_1	下一时隙节点上网络性能总需求	Double	8	$(0, 10^8)$	Mbps

示例: $[y_0, y_1]$

同时为了计算每个任务的引流关联性,应输入作战场景中存在的任务依赖逻辑。任务的依赖逻辑用有向无环的任务依赖关系图表示,用于描述在任务逻辑链中,该任务与其他任务之间触发逻辑关系。在该有向无环图中,可以为边赋予权重来表征任务之间的触发概率,如图 14 所示。

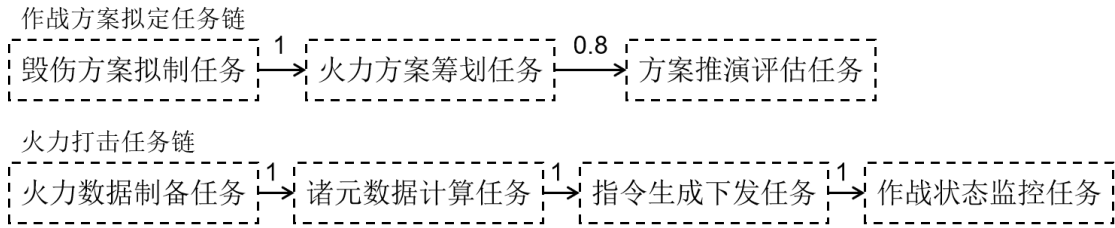


图 14 任务依赖逻辑关系图

任务依赖逻辑关系图用 JSON 文件输入。

JSON 文件格式示例:

```
{
  "作战方案拟定任务链": {
    "毁伤方案拟制任务": {"dependencies": {}},
    "火力方案筹划任务": {"dependencies": {"毁伤方案拟制任务": 1}},
    "方案推演评估任务": {"dependencies": {"火力方案筹划任务": 0.8}},
  },
  "火力打击任务链": {
    "火力数据制备任务": {"dependencies": {}},
    "诸元数据计算任务": {"dependencies": {"火力数据制备任务": 1}},
    "指令生成下发任务": {"dependencies": {"诸元数据计算任务": 1}},
    "作战状态监控任务": {"dependencies": {"指令生成下发任务": 1}}
  }
}
```

(2) 事件驱动模式:

当算法实际部署时,每当有任务到达时进行一次算法调用。算法整体的输入如下表 11 所示,其中 f_0 、 f_1 、 f_2 分别对应该到达任务的特征参数。

表 11. 算法整体输入

输入维度	各维度特征含义	类型	字节数	值域	单位
f_0	该任务的最大时延容忍值	Double	8	$(0, 10^4)$	s
f_1	该任务的传输数据量	Double	8	$(0, 10^4)$	MB
f_2	该任务的处理数据量	Double	8	$(0, 10^8)$	MB

示例: $[f_0, f_1, f_2]$

算法整体的输出如下表 12 所示:

表 12. 预测模型输出维度

输出维度	各维度特征含义	类型	字节数	值域	单位
y_0	执行该任务的算力性能总需求	Double	8	$(0, 10^8)$	FLOPS
y_1	执行该任务的节点上网络性能总需求	Double	8	$(0, 10^8)$	Mbps

示例: $[y_0, y_1]$

3. 2. 5. 2. 算法模型训练阶段的样本格式

为了实现对算法中涉及的 Informer 模型以及结合 LightGBM 的深度神经网络模型的训练, 需要离线样本数据。样本集的格式如下表 13 所示, 需要在上述推理的输入格式上令增任务的执行情况(分配到的算力资源、带宽资源与任务的执行时间)。

由于作战任务以容器的形式在节点上运行。因此任务分配到的算力资源可以通过对应容器的 CPU 利用率乘上 CPU 总算力或者任务分配到的核心数乘上每个内核的算力得到; 任务分配到的网络带宽资源即为容器的(下行)带宽; 任务的执行时间为任务经历的 Wall Clock Time。数据样本集的规模应不少于 30000 条任务数据。

表 13. 训练样本格式

输入维度	各维度特征含义
$f_{task_i,0}$	第 <i>i</i> 个任务的到达时刻
$f_{task_i,1}$	第 <i>i</i> 个任务的最大时延容忍值
$f_{task_i,2}$	第 <i>i</i> 个任务的传输数据量
$f_{task_i,3}$	第 <i>i</i> 个任务的处理数据量
$f_{task_i,4}$	此到达时刻对应的作战阶段
$f_{task_i,5}$	分配给第 <i>i</i> 个任务的算力资源
$f_{task_i,6}$	分配给第 <i>i</i> 个任务的带宽资源
$f_{task_i,7}$	第 <i>i</i> 个任务的执行时间

示例: $[f_{task_0,0}, f_{task_0,1}, f_{task_0,2}, \dots, f_{task_0,7}],$
 $[f_{task_1,0}, f_{task_1,1}, f_{task_1,2}, \dots, f_{task_1,7}],$

$$[f_{task_N,0}, f_{task_N,1}, f_{task_N,2}, \dots, f_{task_N,7}]$$

同样样本数据集中应给出作战场景中存在的任务依赖逻辑。任务的依赖逻辑用表示用 JSON 文件输入。

JSON 文件格式示例：

```
{
  "作战方案拟定任务链": {
    "毁伤方案拟制任务": {"dependencies": {}},
    "火力方案筹划任务": {"dependencies": {"毁伤方案拟制任务": 1}},
    "方案推演评估任务": {"dependencies": {"火力方案筹划任务": 0.8}}},
  "火力打击任务链": {
    "火力数据制备任务": {"dependencies": {}},
    "诸元数据计算任务": {"dependencies": {"火力数据制备任务": 1}},
    "指令生成下发任务": {"dependencies": {"诸元数据计算任务": 1}},
    "作战状态监控任务": {"dependencies": {"指令生成下发任务": 1}}}
}
```

3.2.6. 算法整体流程

为了响应动态环境下面向任务的资源需求感知和面向节点的资源状态预测两种不同的需求，优化系统资源分配和任务调度的效率，该算法支持事件驱动和时间驱动两种触发模式。时间驱动模式则以设定的时间间隔触发算法，用以识别全局任务负载的长期变化趋势和资源节点的状态周期性演变，对下一时隙的节点资源负载进行预测，以衡量节点后续承担算力任务的能力。事件驱动模式主要针对节点上实时到达的任务请求进行执行该任务对于算网资源需求的感知，以适配动态任务调度的实时性要求。这两种触发模式协同配合，兼顾了资源调度的短期响应灵活性和长期规划的全局最优性。

面向异构算网资源的智能感知与预测算法的两种触发模式对应的流程如图 15 所示，其中时间驱动的流程用黑色粗实线表示，事件驱动的流程用蓝色粗实线表示。

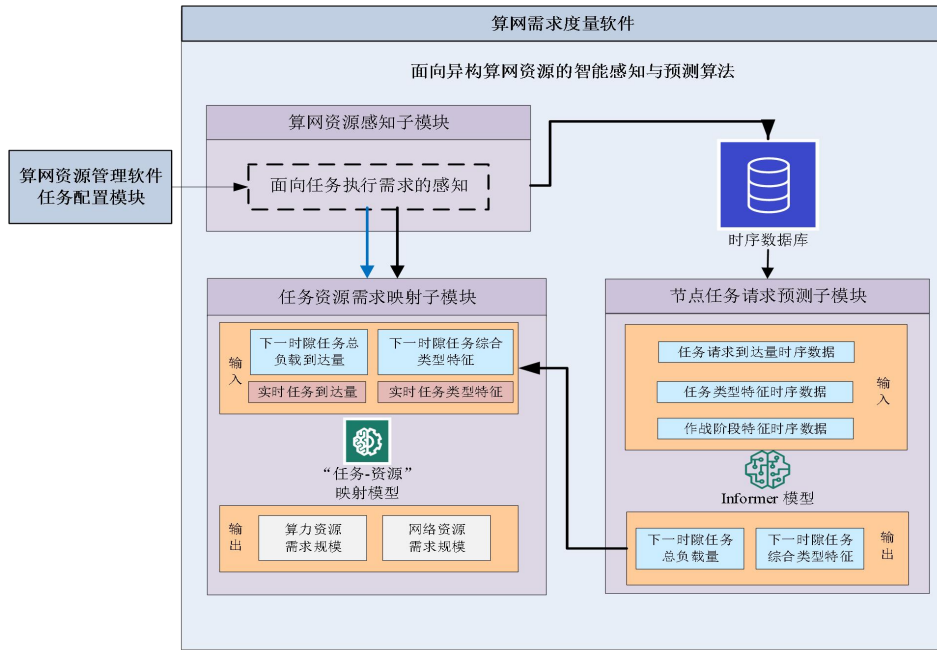


图 15. 算法流程图

(1) 时间驱动模式

步骤一：每个预测时隙中，算网资源感知模块从算网资源管理软件中的算网任务配置模块中接受对于到达任务执行需求的感知数据，存入节点的数据库中。

步骤二：节点任务请求预测子模块基于本地节点数据库中的历史时序数据，对上一时隙内的数据进行归纳处理，输入到 **Informer** 模型中对下一时隙节点上将到达的任务总体负载量、任务综合类型特征向量进行预测。

步骤三：任务需求映射子模块上部署“任务-算力资源”映射模型，接受节点任务请求预测子模块预测所得的预测负载量和任务综合类型特征向量作为映射模型的输入，实现对下一时隙节点算网总资源需求的感知。

(2) 事件驱动模式

步骤一：当新任务请求到达节点时，算网资源感知子模块根据任务的配置信息，对该请求进行任务类型特征向量表征，并将任务到达量与任务类型特征向量发送给任务需求映射子模块作为“任务-资源需求”映射模型的输入数据。

步骤二：任务需求映射子模块上部署“任务-算力资源”映射模型，接受算网资源管理子模块输出的任务到达量和任务综合类型特征向量作为映射模型的输入，实现对下一时隙节点算网总资源需求的解析。

3.3. 通用质量特性要求

- 算力网络资源负载预测准确率不低于 80%；
- 支持不少于 3 种算力网络资源的表征,支撑的算网资源调度管理节点数量不低于 1000 个。

3.4. 评估标准

为验证面向异构算网资源的智能感知与预测算法的有效性和优越性,需从预测精度、实时性和可拓展性三个维度设计评估标准。通过量化指标与对比实验,验证算法在复杂战场环境下的综合性能。

- (1) 以时间驱动方式触发的算法执行时间 (s)；
- (2) 以事件驱动方式触发的算法执行时间 (s)；
- (3) 均方误差 (MSE)：

$$MSE = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

- y_i : 任务实际所需的算力资源或网络资源规模 (FLOPS 或 Mbps)。
- $\hat{y}_i$: 算法预测的任务算力资源或网络资源规模需求值 (FLOPS 或 Mbps)。
- N : 测试样本总数。

应用场景：评估模型对不同类型任务（如信号级仿真和时敏目标识别任务的算网需求预测精度。

- (2) 平均绝对误差 (MAE)：

$$MAE = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|$$

- y_i : 任务实际所需的算力资源或网络资源规模 (FLOPS 或 Mbps)。
- $\hat{y}_i$: 算法预测的任务算力资源或网络资源规模需求值 (FLOPS 或 Mbps)。
- N : 测试样本总数。

应用场景：评估模型对不同类型任务（如信号级仿真和时敏目标识别任务的算网需求预测精度。

3.5. 验证方案

3.5.1. 验证要求

3.5.1.1. 验证指标

MSE 和 MAE 的目标值应尽可能小,表示预测值与实际值之间的误差较小。

目标值的具体大小取决于任务的计算资源需求范围(如 FLOPS 或 Mbps 的数量级)。如果任务的资源需求波动较大,允许的误差范围可以适当放宽;如果任务对资源需求预测精度要求较高,则目标值应更严格。

参考设置:

- **MSE:** 目标值可以设置为实际需求值的 5%-10% 的平方。例如,如果任务的资源需求在 1000 FLOPS 左右, MSE 目标值可以设置为 10000。
- **MAE:** 目标值可以设置为实际需求值的 5%-10%。例如,如果任务的资源需求在 1000 FLOPS 左右, MAE 目标值可以设置为 50-100。
- **Sync_num:** 支持同时对于不少于 5 类任务进行资源需求感知的进程数,要求算法多进程的运行环境下,仍保证实时性与准确性。
- **Time_model1:** 采用时间驱动执行模式下的运算时间不超过 5 秒。
- **Time_mode2:** 采用事件驱动执行模式下的运算时间不超过 5 秒。

指标	目标值	适用场景
算力需求 MSE	≤ 10000	针对信号级仿真等高计算量任务的算力需求规模拟合
网络需求 MSE	≤ 25	针对时敏目标识别等低时延任务的网络需求规模拟合
算力需求 MAE	≤ 100	针对信号级仿真等高计算量任务的算力需求规模拟合
网络需求 MAE	≤ 5	针对时敏目标识别等低时延任务的网络需求规模拟合
并行进程数 Sync_num	≥ 5	针对突发任务或任务密集场景下算法的并行调用能力
时间驱动-算法执行时间 Time_model1	≤ 5	时间驱动模式下的算法执行时间
事件驱动-算法执行时间 Time_mode2	≤ 5	事件驱动模式下的算法执行时间

3.5.1.2. 验证方式

3.5.1.2.1. 仿真环境搭建

a) 平台架构: 基于 Python 3.8 + PyTorch 1.12 构建分布式仿真平台, 模拟 50 个移动节点(装甲车)的战场环境。

b) 动态场景: 在仿真环境搭建时, 将战场态势分为战备值班阶段、作战准备阶段、火力打击阶段和毁伤评估阶段。并在每个作战阶段内周期性地注入不同的作战任务, 同时不定期增加突发性的任务请求, 如临机打击任务。

任务执行过程中的实际浮点计算次数，可以通过在典型目标环境上运行任务，利用性能分析工具（如 Linux 的 perf、Intel VTune、NVIDIA NSight 等），统计浮点指令的执行数量。多数处理器的性能计数器能够直接记录单精度（FP32）、双精度（FP64）等浮点指令的累计执行次数。对于规模较大的任务，可测量代表性子任务的浮点操作次数，并按比例估算整体任务的浮点计算量。不同阶段的务到达设置如表 14 所示：

表 14 算法验证任务到达流程

阶段	任务链	任务块	浮点计算次数 (1e3FLOPS)	传输数 据量 (MB)	最大时延 容忍 (S)	子服务触发时机
战备 值班	目标情报处理 任务链	目标情报 处理任务	800 - 20,000	1 - 10	0.1-1	5min 为周期到达
	作战状态 监控任务链	作战状态 监控任务	1 - 5	0.05 - 0.5	0.1-1	5 min 为周期到达
作战 准备	作战方 案拟定 任务链	毁伤方案 拟制任务	50 - 300	0.02 - 0.2	0.5-5	24h 为周期到达
		火力方案 筹划任务	600 - 4,000	0.04 - 2.0	5-20	上一步骤结束后立即执行
		方案推演 评估任务	2,000 - 10,000	0.05 - 2.0	1-10	上一步骤结束后 0.5-1h 内随机执行，有 20%概率不执行
火力 打击	火力打 击任务 链	火力数据 制备任务	5 - 30	0.01 - 5.0	1-10	以 2h 为周期到达
		诸元数据 计算任务	50 - 300	0.001 - 0.01	0.1-0.5	上一步骤结束后立即执行
		指令生成 下发任务	0.3 - 2	0.001 - 0.01	0.1-0.5	上一步骤结束后立即执行
		作战状态 监控任务	1 - 5	0.05 - 0.5	0.1-1	上一步骤结束后 5-10min 内随机执行
毁伤 评估	目标毁 伤评估 任务链	目标毁伤 评估任务	0.05 - 0.3	1 - 50	0.05-0.5	集火打击结束后 10-30min 内

3.5.1.2.2. 数据输入规范

a) 训练集：80%数据通过 NS-3 网络仿真生成，涵盖高负载（峰值 $\geq 2000\text{MB}$ ）、低连通（加权重 ≤ 5 ）等极端场景。

b) 测试集：20%数据引入真实战场日志（脱敏处理），验证算法迁移能力。

3.5.1.2.3. 对比实验设计

a) 基线模型：ARIMA、LSTM、Prophet。

b) 评估流程：固定随机种子（seed=42），重复实验 5 次取均值，记录 MSE、Acc 及响应时间。

3.5.1.3. 验证条件

a) 深度学习框架：PyTorch 1.12 + CUDA 11.6。

b) 辅助工具：LightGBM 3.3（特征筛选）、Scikit-learn 1.2（数据预处理）。

3.5.2. 训练样本仿真生成

本项目的训练样本采用 Google Cluster Trace 2019 数据集。Google Cluster Trace 2019 数据集提供了丰富的集群任务日志和节点资源状态信息，能够为节点任务请求预测子模块提供有效的验证支持。该数据集包含任务历史负载到达序列（如任务提交时间、运行时长、资源请求）、节点资源使用情况（如 CPU、内存占用率）、任务依赖关系（如 DAG 结构）等多维时序数据，可与子模块中的 Informer 模型结合，挖掘任务负载与资源状态的关联性。通过将任务到达序列与节点资源状态映射为输入特征，模型可预测未来时间槽的任务综合负载（如任务数量、资源需求峰值）。尽管数据集未直接包含战场特征或地理环境信息，但其高分辨率的任务与资源动态数据可为多维特征建模提供基础，同时可通过特征扩展（如引入周期性任务模式或异常事件标记）模拟战场场景下的复杂性，从而验证模型在真实集群环境中的预测能力与鲁棒性。

3.5.3. 验证项目

序号	试验项目	试验目的	试验内容
1	算法准确性验证	验证节点资源需求的准确性	实验中利用算法在测试集上验证节点算网资源需求预测的准确性
2	算法可拓展性验证	测试算法在高并发场景下的性能	调整同一时间窗口内节点上到达的不同任务类型数量，记录算法在多任务并行处理中的性能表现
3	算法计算效	验证算法的实时性	实验中记录算法在不同并行进程数

序号	试验项目	试验目的	试验内容
	率验证		下采用不同驱动触发方式的运行时间，评估算法的计算效率

4. 技术指标满足情况分析

序号	技术指标	满足情况	技术支持
1	算力需求 MSE	≤ 10000	基于树的深度神经网络模型支持对于任务特征的深度捕捉学习
2	网络需求 MSE	≤ 25	基于树的深度神经网络模型支持对于任务特征的深度捕捉学习
3	算力需求 MAE	≤ 100	基于树的深度神经网络模型支持对于任务特征的深度捕捉学习
4	网络需求 MAE	≤ 5	基于树的深度神经网络模型支持对于任务特征的深度捕捉学习
5	时间驱动-算法执行时间 Time_model1	≤ 5	Informer 模型引入概率稀疏自注意力机制和生成式解码器设计，降低了计算复杂度
6	事件驱动-算法执行时间 Time_mode2	≤ 5	Informer 模型引入概率稀疏自注意力机制和生成式解码器设计，降低了计算复杂度
7	时间驱动-算法执行时间	≥ 5	算法通过对不同类型任务特征进行细粒度描述，支持对多种任务进行高效的资源需求感知

5. 研究进度、预期研究成果及应用方向

5.1. 研究进度

年度	主要研究内容	完成时间	完成形式
2025 年	调研分析算力网络资源负载预测的最新研究成果与研究异构算力资源统一度量模型	2024.12.30	形成调研报告
	研究分布式算网资源与业务需求感知技术与研发面向异构算网资源的智能感知与预测仿真实例。	2025.01.30	形成调研报告
	调研主流时序序列预测算法，进行算法选型工作	2025.02.30	形成算法选型论证报告
	完成面向异构算网资源的智能感知与预测算法开题	2025.02.30	形成算法开题报告
	推进面向异构算网资源的智能感知与预测算法的设计报告整改	2025.05.30	形成算法中期报告
	完成面向异构算网资源的智能感知与预测算法仿真代码编写	2025.09.30	试验验证、仿真

	完成面向异构算网资源的智能感知与预测仿真实例，并在实装上进行测试与验证	2025.10.15	试验验证、仿真、专家评审
	进行结项工作，算法结题汇报	2025.12.01	提交结题报告及实物类成果

5.2. 预期研究成果

1) 报告类成果

- 面向异构算网资源的智能感知与预测算法设计说明
- 面向异构算网资源的智能感知与预测算法模型测试说明
- 面向异构算网资源的智能感知与预测算法模型测试报告
- 面向异构算网资源的智能感知与预测算法模型使用说明
- 面向异构算网资源的智能感知与预测算法模型研制总结报告

2) 实物类成果

- 算法软件、训练模型及配套显示界面(含源代码+可执行程序);

5.3. 应用方向分析

5.3.1. 典型任务 1：信号级仿真任务

信号级仿真任务具有计算密集型特征，其模型复杂度高、单次执行耗时长，对节点的算力资源（如 CPU、内存）和存储资源（如磁盘空间）需求极大，且任务到达模式相对低频但突发性强。传统静态调度策略难以动态感知节点的实时负载与资源瓶颈，易导致任务积压或节点过载。在此场景下，本算法基于历史运行状态信息，可对节点未来时隙的任务请求量进行精准预测，并对节点未来的负载变化趋势进行评估，从而提前识别高负载节点的资源瓶颈风险。例如，当预测到某节点即将因磁盘使用率超限而资源耗竭时，算法可触发任务迁移机制，将部分仿真任务卸载至邻近空闲节点，避免任务中断。

5.3.2. 典型任务 2：时敏目标识别任务

在时敏目标识别任务场景中，算法的实时性与动态调度能力成为关键支撑。此类任务具有强突发性、高时效性特点，任务到达频率波动大，且对端到端处理时延要求极为苛刻。传统调度策略因缺乏负载预测能力，难以应对瞬时任务洪峰，易引发节点过载和响应超时。本算法利用 **Informer** 模型的长序列建模优势，可以快速捕捉任务到达的短期规律（如作战阶段切换导致的识别请求激增），并结合

节点实时算网资源状态，预测下一时隙的任务综合负载量。例如，在进攻阶段预测到目标识别任务量将激增时，算法可触发邻近低时延节点的算力协同，确保任务快速分发。

6. 保障条件及措施

6.1. 研发团队保障条件

项目研发团队由刘韵洁院士领衔，获中国电子学会创新团队奖，是北京邮电大学网络与交换技术全国重点实验室的核心力量。目前该团队已建立起了由院士、教授、副教授、讲师、博士后组成的层次分明的科研团队。团队在未来网络架构体系、能源互联网、云网融合与算力网络等方面具有深厚的研究基础，近 5 年来，在 IEEEJSAC、IEEETWC、ACMSigcomm 等国际顶级期刊会议累计发表 SCI/EI 论文 500 余篇，获授权发明专利 100 余项，国际授权 4 项，出版专著 15 部，软件著作权 50 余项。团队先后牵头承担了 1 项国家重大科技基础设施工程、1 项国家自然科学基金重大研究计划项目、3 项国家重点研发计划项目、5 项国家重点研发计划课题、1 项 973 项目、1 项国家 863 项目、2 项国家科技重大专项、10 项国家自然科学基金等 30 余项国家级项目以及 80 余项省部级与校企合作项目。成果先后获 2023 年中国通信学会科技进步特等奖、2023 年北京市自然科学二等奖、2023 年中国通信学会科技进步一等奖、2017 年中国通信学会技术发明一等奖、2021 年中国航天科技集团科技进步二等奖，以及入选中国科协 2021 年“科创中国”先导技术榜单、2020 年世界互联网大会“世界互联网领先科技成果”、中国通信学会 2021 年未来网络领先创新科技成果等。

项目研发团队长期从事通信网、计算机网络体系结构及其关键技术研究，工作基础充分，科研条件完备，具有先进的仪器和较完善的通信实验仪器、开发平台，包括：高档工作站、服务器、高性能计算机、Cisco 路由器、HP 交换机、NetFPGA 板卡以及微型计算机、通信协议分析仪、传输测试仪、网络分析仪、误码测试仪等仪器；另外还有 NS3、Qualnet 等网络仿真软件，可为项目的顺利开展提供良好的工作支撑条件。此外，项目研发团队承担了信息与通信领域首项国家重大科技基础设施项目“未来网络试验设施（CENI，总投资 15.14 亿元）”的建设，已建设有覆盖全国 40 个城市、133 个边缘网络的基础设施，可支持一至七层各类技术的创新研究，可满足本项目等多种场景的试验验证需求。

6.2. 质量保障措施

研究团队将严格按照合同或计划任务书的要求完成项目目标任务。在项目前期，团队将充分进行调研和规划，准确获取项目的技术指标、需求及交付标准，并确保双方沟通清晰无误。调研过程中将进行多次确认，组织团队输出深入、完整且全面的调研报告与分析，确保项目规划准确无误并满足需求。

为确保项目交付质量，在整个项目执行过程中将实施严格的质量管理和测试机制，包括功能测试、性能测试和安全性测试等环节，同时按要求提交项目执行报告、成果总结报告及项目经费预算等相关材料，并参与定期检查或评估，确保项目全程透明、科学和高效。研究团队将遵守学术规范和职业道德，保证项目的科学性和创新性，确保输出结果的高质量与严谨性。

在项目交付后，将建立高效的沟通渠道，通过电话、邮件或其他约定的方式提供及时的技术支持与解答。同时，设立专门的技术支持团队，负责后续的技术服务和用户培训。该团队将及时收集反馈意见，处理突发技术问题，包括软件错误修复、故障排除以及产品性能优化等，以全面保障项目成果的稳定使用和持续改进。

6.3. 保密工作措施

在整个项目过程中，项目团队将严格遵守并执行相关的保密法规和协议，同时针对项目团队中学生参与的特殊情况制定详细管理方案。具体如下：

6.3.1. 内部任务分解与人员分工

坚持“涉密工作由涉密人员完成，非涉密学生仅参与非涉密环节”的总体原则，对项目任务进行分解和标识。按照“按需知密、最小授权”原则，严格限定项目成员可接触的信息范围和权限等级。对所有任务在立项阶段进行保密属性评估，明确标注为“涉密任务”“非涉密任务”或“边界任务”（可能涉及保密内容的接口任务），并建立任务清单和责任台账。

（1）涉密人员工作内容

涉密人员为已完成涉密审查并具备相应保密资格的科研人员和管理人员，其主要承担以下工作：

- 涉密方案与核心技术研发：包括项目总体方案设计、核心算法与关键技术研究、涉密技术路线论证及决策等。

- 涉密数据采集与处理：承担涉密原始数据的采集、预处理、分析建模和结果研判，确保涉密数据不向非授权人员扩散。
- 涉密文件编制与审定：负责撰写、编辑和审定涉密技术报告、设计文档、内部评估材料及对上级报送的涉密材料，对内容进行密级标注和敏感点把关。

（2）非涉密学生工作内容

非涉密学生不纳入涉密人员管理，其参与范围严格限定在经筛选的非涉密任务中，主要包括：

- 一般性实验与仿真：在提供的脱敏或虚拟样本数据、公开数据集基础上，开展算法验证、参数调优、对比实验等工作，不接触真实涉密数据。
- 辅助性分析与整理：协助涉密人员对经脱密处理后的统计结果进行整理、可视化展示和一般性规律分析，避免涉及数据来源、关键指标阈值等敏感内容。
- 非涉密文献调研与综述：围绕公开领域的基础理论、通用技术路线、公开标准和行业发展现状进行检索、阅读和撰写综述，不涉及项目内部技术细节和专有方法。

6.3.2. 岗前培训与签署保密责任书

对所有参与项目的学生和工作人员，均需进行入职及项目相关的保密培训，确保其充分了解保密规定和要求。培训内容包括保密法规、项目相关的文件管理规定、访问权限规则等。所有团队成员均需签署保密协议，与学生单独签订《非涉密参与保证书》，并对协议履行情况设立监督机制。

6.3.3. 涉密场所与办公区域管理

（1）区域划分与物理隔离：项目办公区域实行“三区管理”：涉密区、受控区和普通办公区。涉密区实行封闭管理，受控区为涉密前后过渡区域，普通区为学生及非涉密人员主要活动区域。涉密工作（含涉密文件阅读、涉密讨论、涉密设备操作）必须在涉密区内进行，禁止在开放实验室、公共自习区域或外部场所开展。通过门禁系统、视频监控和区域标识，对涉密区进行物理隔离，未经授权人员不得进入。

（2）人员出入与行为规范：涉密区实行门禁刷卡和访客登记制度，学生一

般不得进入涉密区，确因工作需要进入受控区的，须由涉密人员全程陪同并记录理由和时间。涉密区内严禁携带个人手机、平板、U 盘等可记录、存储或无线通信的电子设备，确需使用的设备须通过保密审查和登记。在涉密和受控区域内，禁止随意谈论项目核心技术内容，会议和讨论需在指定会议室进行，按规定做好会议记录和资料回收。

6.3.4. 涉密文件与资料管理流程

（1）涉密文件全生命周期管理

涉密文件由涉密人员编制，按照内容敏感程度标注密级和保密期限，并填写《涉密文件登记表》，并统一由项目保密秘书进行编号、登记和建账，记录文件名称、编号、密级、份数、保管人和使用范围等信息。纸质涉密文件存放于专用保密柜内，实行双人双锁管理；电子文件存储在涉密服务器或加密硬盘中，通过严格账号和权限控制。

（2）涉密文件使用与流转控制

涉密文件仅限授权涉密人员在涉密场所内查阅，严禁外借、拍照、复印和私自携带出涉密区域。确需在项目内部有限流转时，由保密秘书办理传阅手续，对传阅人、时间和范围进行登记，并在规定期限内收回。学生不得接触任何标注为涉密的文件，涉密内容如需间接体现给学生，必须由涉密人员进行脱敏、归纳后以非涉密形式提供。

（3）电子文件管理与打印控制

部署物理与逻辑隔离的涉密网络系统，与非涉密办公网络严格分离，涉密网络仅供涉密人员使用，学生账号不得接入。同时对涉密电子文件采用强加密算法进行保护，并设置访问日志和操作审计功能，防止非法复制和外传。涉密文件仅允许在指定的保密打印设备上打印，打印任务须由涉密人员操作，系统自动记录打印人、时间和份数，打印后的废页、样张按涉密文件处理。

（4）涉密文档销毁

对失效、作废或经审批同意销毁的涉密文件，统一使用专业保密文档销毁机进行物理粉碎，做到不可恢复。销毁过程由两名以上涉密人员现场见证，并填写《涉密文件销毁记录表》，列明销毁文件的种类、编号、密级和数量。对存储涉密信息的电子介质，在报废前按单位规定执行数据清除或物理销毁操作。

6.3.5. 保密检查与应急预案

(1) 定期检查与风险排查

建立定期保密检查机制，由项目负责人牵头、保密秘书及相关涉密人员参与，对人员管理、场所管理、文件管理和设备使用进行全面排查。重点检查学生在执行非涉密任务时是否存在违规存储、违规拷贝、跨区域传输等行为，并对发现的问题及时整改。检查情况形成书面报告，作为项目阶段性评估和总结的重要内容。

(2) 突发事件处置与责任追究

一旦发现可能存在泄密隐患或违规操作行为，立即启动项目保密应急预案：包括紧急断网、封存设备、停止相关任务、召回资料等措施。对已发生的泄密或严重违规行为，及时上报单位保密部门，配合开展调查，查明原因和影响范围，按规定追究相关责任人的法律和纪律责任。根据事件教训及时修订和完善相关管理制度、技术措施和培训内容，防止类似问题再次发生。

7. 承担研究任务主要人员概况

姓 名	职务/职称	单 位	参与周期	任务分工
谢人超	教授	北京邮电大学	任务全周期	项目总负责人，全面负责研究整体规划、技术路线、科研统筹管理、理论指导等。
黄韬	教授	教授	任务全周期	协助项目组织实施，负责为算法理论研究提供技术支持
唐琴琴	副研究员	北京邮电大学	任务全周期	协助项目组织实施，负责为算法优化模型提供技术支持
冯立	博士	北京邮电大学	任务全周期	负责算法选型调研与分析
冯瑞琪	博士	北京邮电大学	任务前期	负责算法选型调研与分析
徐宏亮	博士	北京邮电大学	任务前期	数据采集与处理，完成任务负载预测模型训练与数据预处理模块开发。
许德昀	硕士	北京邮电大学	任务前期与中期	算法模块设计与研发
邓喆	硕士	北京邮电大学	任务前期与中期	算法模块设计与研发
周弘烨	硕士	北京邮电大学	任务前期与中期	算法模块设计与研发

张语嫣	硕士	北京邮电大学	任务中期	实现仿真实验环境搭建，优化实验数据记录与问题反馈机制。
万卓然	硕士	北京邮电大学	任务中期	数据实验结果可视化，完成报告撰写与模型结果核验。
李敏宽	硕士	北京邮电大学	任务中期与后期	协助完成数据分析与实验验证，辅助模型调优和实验结果记录。
李妃	硕士	北京邮电大学	任务后期	负责日志记录系统开发，完成实验问题的在线诊断和维护支持。

8. 技术风险分析

（1）多源异构数据整合与分析挑战

- 产生原因：数据来源广泛且格式多样，包括传感器数据、网络数据、历史数据和实时数据，涉及结构化和非结构化数据。数据质量不一致、传输延迟以及数据处理开销大，增加了整合难度。
- 导致现象：数据整合不及时或不准确，导致感知和预测误差，影响资源分配的准确性和系统的实时性。
- 定位分析：分析数据采集、清洗和传输过程中的瓶颈，检查数据质量问题（如缺失值、异常值），确定标准化方式。
- 解决措施：设计统一的数据标准和接口，开发高效的数据清洗和整合算法；采用分布式数据处理框架（如 Spark）以提高处理效率；引入边缘计算技术减少数据传输延迟；对关键数据源进行质量监控和实时校验。

（2）算网资源动态协同难题

- 产生原因：算网资源的异构性（如计算资源、存储资源、网络资源）和分布性增加了管理复杂性，且缺乏统一的资源描述标准，导致资源感知和调度的复杂性增加。
- 导致现象：模型训练过拟合，泛化性欠佳，部署到节点上时的性能有所差异。
- 定位分析：分析节点的资源状态数据（如 CPU 利用率、内存使用率、网

络带宽等），识别资源描述不一致的问题。在模型训练层面，评估模型在不同节点上的性能差异，分析过拟合和泛化能力不足的原因。

- 解决措施：设计统一的算力资源度量体系，将异构资源的性能指标标准化，通过算网资源感知模块，实时监控节点的资源状态，并将数据存入数据库，为后续调度提供支持。针对模型训练的过拟合问题，引入增量学习技术，使模型能够在不同节点上快速适配，提升泛化能力。

（3）算法模型动态适配压力

- 产生原因：战场环境的动态变化（如任务类型、负载模式、节点状态）会导致算法模型的预测能力逐渐衰减。离线训练的模型难以适应实时变化，且大规模训练开销高。
- 导致现象：算法预测精度下降，导致资源分配和调度效果变差，可能引发任务失败或系统过载。
- 定位分析：检查模型预测误差的变化趋势，分析模型参数是否需要调整；评估实时数据对模型的影响，识别模型适应性不足的场景。
- 解决措施：支持模型的在线微调；设计自适应模型更新机制，结合实时数据动态调整参数；采用迁移学习技术减少训练开销；对模型性能进行定期评估和优化，确保其适应性和鲁棒性。

（4）多目标性能指标冲突

- 产生原因：准确性、实时性、可靠性、可扩展性等性能指标之间存在权衡关系。例如，提高预测准确性可能需要增加神经网络的层数与参数规模，增加计算的复杂度。
- 导致现象：系统在多目标优化中部分性能指标下降。
- 定位分析：分析各性能指标之间的依赖关系和冲突点，评估不同优化策略对系统整体性能的影响；识别资源分配失衡的场景（如高负载节点或低优先级任务）
- 解决措施：；开发自适应调度算法，结合实时监控动态优化系统性能；通过仿真测试验证优化效果。

9. 经费概算

本项目经费概算如下表所示。具体费用包括设备费、材料费、业务费、劳务

费、差旅费以及知识产权及成果推广费。

序号	条款名称	金额（万元）
1	科学基金资助项目直接费用合计	60.00
2	设备费（研发所需设备购置与租赁）	15.00
3	材料费（耗材、研究所需试剂购买等）	8.00
4	业务费（调研、实验测试及第三方协作）	17.00
5	劳务费（参与人员补助、博士后津贴等）	15.00
6	差旅费（会议、课题调研）	5.00